

Unless otherwise stated, copyright © 2025 The Open University, all rights reserved.

Printable page generated Tuesday 2 December 2025 at 14:01

Block 2 Part 1 Programming with Scratch

Introduction

This is the first part of the programming block of TM111. In this part, you will be introduced to:

- some fundamental concepts in programming
- some key features of Scratch programs and of the Scratch programming environment
- how to create small Scratch programs to accomplish specific tasks.

We aim to give you a broad overview of the capabilities of Scratch as well as introducing some important ideas that will be developed further throughout this block.

In Sections 1.1 to 1.4 of this part, you will create some small programs which use visual and audio features of Scratch as well as some programs that involve simple input and output. You will learn how to make a program respond to user actions, such as pressing a keyboard key, and will also begin to understand how different parts of a program may interact with each other.

Section 1.5 then provides you with further practice with these ideas through undertaking an exercise.

Introduction to Scratch

In the first activity, you will run two small Scratch programs to introduce you to some key facts about Scratch programs. The instructions have been written using the Scratch desktop app but if you are using the online version there might be very small variations in the way some items appear. However, most of the screens are the same regardless of which version you are using.

Activity 1.1 (exploratory)

(a)

Launch Scratch. Open the project `Activity_1.1a.sb3` by selecting 'File' at the top of the screen and then 'Load from your computer', and then navigating to the subfolder `Part 1` within the folder `TM111 Block 2 Projects`.

(Alternatively, if you do not already have the Scratch desktop app open, then you could open a Scratch file by double-clicking on the filename in the folder of Scratch files. Note that you may need to associate the `.sb3` file extension with the desktop app; see the relevant Scratch FAQ on the module website for more information. If you are using the online version of Scratch, then you have to use the 'File' method described above.)

This project contains a program that allows the user to play a game we will call the Octopus Hunt. Now follow these instructions.

1. Click on the large screen icon, as shown in Figure 1.1, at the top right of the Scratch window to move into full-screen mode.



Figure 1.1

2. Click on the green flag icon, as shown in Figure 1.2(a), at the top left of the full screen to run the program.
3. Move your mouse pointer (cursor) around and the octopus will follow it. The octopus will simply follow your mouse pointer – don't try to drag it. Try to keep the octopus away from the shark.
4. Click on the red stop icon, as shown in Figure 1.2(b), to stop the program.
5. Then return to the main Scratch window by clicking on the small screen icon, as shown in Figure 1.2(c), at the top right of the screen.

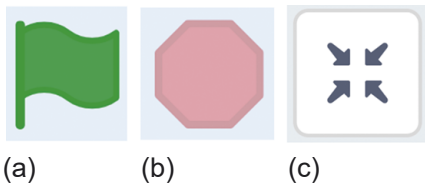


Figure 1.2

The full-screen mode provides a view of the program as a user would typically see it, without any code visible. Now you are back in the programming environment, where you will later explore programs and create code.

Note that in the programming environment there are also the same green flag and red stop icons towards the top of the screen, and these allow you to run and to stop the program as before; the visual effects are shown in the smaller screen. Try this.

(b)

Next open the file `Activity_1.1b.sb3` by selecting 'File' and 'Load from your computer' and then navigating to this file. Opening another Scratch file closes the current one. If any changes were made to the previous file, Scratch will present you with the prompt: 'Replace contents of the current project?'. You should click 'OK' to continue loading the new project. Any changes to the previous file will be discarded unless you save them to your computer, which we will cover in Activity 1.3.

This program allows the user to undertake a quiz that we will refer to as the Spelling Tutor. In either full-screen mode or within the programming environment, start the program with the green flag icon and follow the instructions. Click on the Scrabble tile image to start the quiz, and note that the user may be asked to spell the same word more than once. Close the Scratch environment by clicking on the cross icon at the top right of the window.

These programs illustrate some important facts:

- There are several ways for the user to make a Scratch program do something:
 - Clicking the green flag icon. This is conventionally (but not always) the way a Scratch program is started.
 - Clicking on an image on the screen – for example, the Scrabble tile image in Spelling Tutor.
 - Pressing a specific keyboard key – for example, the spacebar in Spelling Tutor.
 - Clicking the red stop icon to stop a running program. This is only necessary if a program runs indefinitely, such as Octopus Hunt. Other programs may simply end.
- Different parts of a Scratch program may be in action simultaneously. For example, in Octopus Hunt, both the octopus and the shark move at the same time.
- Parts of a Scratch program may interact with each other. For example, in Octopus Hunt, when the shark catches up with the octopus, the shark appears to 'eat' the octopus, and the octopus briefly disappears. Or, parts of a program may interact with the user: again in Octopus Hunt, the octopus follows the user's mouse pointer.
- The tasks carried out by Scratch programs are varied. A program may or may not:
 - require input – that is, data from the user
 - produce textual output – that is, data to the user
 - produce visual results – for example, characters or objects moving about
 - produce audible results – for example, music playing
 - run for a very short period of time; it may run for longer, or indefinitely.

In Section 1.1, you will start to look at the code that forms Scratch programs, and from time to time will reflect back on these facts.

1.1 Doing things in sequence

A Scratch program typically involves one or more **sprites**. A sprite is a distinct component of a program, whose role in the program – what it does – is specified by creating code for that sprite. A sprite has one or more graphics, or images, each called a **costume**, which is what the sprite 'looks' like at any one time. A sprite can be made to move around, 'speak' via speech bubbles, make sounds, change costume, interact with other sprites and with its environment, and more, according to the sprite's code.

For example, in the Octopus Hunt program the shark is implemented by a sprite called Shark that has three costumes, depicting the sprite in different poses, as shown in Figure 1.3. It is these costumes that are used, in ways you will learn about shortly, to make the shark appear animated.



Figure 1.3

Shark's code directs it to move around, reacting appropriately when it encounters the octopus, which is implemented by a sprite called Octopus.

A Scratch program may contain any number of sprites, but only one **stage**. The stage is a component whose purpose is often just to provide suitable background images for programs, known as **backdrops** – for example, the underwater scene in Octopus Hunt. Code may be created for the stage, but in many programs, including those you will meet in TM111, the stage has no code.

You might notice a theatre metaphor at play here in the use of terms such as *stage*, *backdrop* and *costume*. Creating a Scratch program can have similarities with putting on a play in a theatre: the scene is set on the stage via backdrops and the characters (sprites) act out roles wearing various costumes. The programmer is rather like the theatre director: setting the scene and (as you will shortly see) orchestrating the actions of the sprites via chunks of code – and, in fact, you'll see that these code chunks are known as *scripts*.

1.1.1 Speech and sounds

Have the *Scratch quick reference* to hand as you work through the next activity. This highlights the main features of the environment, and you should consult it as necessary as you work through Block 2.

Activity 1.2 (exploratory)



Watch Video 1.1, which introduces the Scratch programming environment and demonstrates how to construct a small program. Some of the ideas will be revisited in Activity 1.3, and you might want to watch this video again during that activity if you are unclear about any aspect.

In the next activity, you will learn how to make a sprite emit a speech bubble to display text and how to make a sprite produce a sound.

Activity 1.3 (exploratory)

Open the project `Activity_1.3.sb3` by selecting 'File' at the top of the screen and then 'Load from your computer'.

Each Scratch program is housed in a **project**. A project contains not only the program code but also resources used by the program, such as graphics and sounds. What you see initially is the *default* project; this contains a sprite, Owl, which has costumes and a sound but no code.

Scratch files may be referred to as Scratch projects.

Saving a project

You are going to create a project of your own. Whenever you do this, you should start by naming the project and saving it, so that you can re-save it from time to time as you work, in case something goes wrong and you want to revert to the previous version. Use the 'File' menu and the 'Save to your computer' option to save this project as `Activity_1.3_sol.sb3` (standing for Activity 1.3 solution) in the folder Part 1.

The program specification

In this activity, you will create a program involving the sprite, Owl. Since Owl comes equipped with costumes and a sound ready to use, it is simplest to start programming with this sprite before moving on to others.

We will first guide you through the steps involved in creating a program to make Owl do the following whenever the user clicks on the green flag icon:

- welcome the user to TM111
- make a 'hoot' sound
- introduce itself to the user as the Scratch owl.

Such a description of the tasks to be carried out by a program is called a **specification**.

If you have prior programming experience, or are simply feeling confident, then you might like to see whether you can achieve this by yourself now. In fact, at any point in any activity you should feel free to see if you can achieve what is required by yourself. But (1) don't waste time if you can't work it out, and (2) always return and read through the complete activity, since it will include discussion of concepts and techniques that you must be aware of.

(a)

Ensure Owl is selected (use the thumbnail in the bottom right grey pane) with its Code pane to the fore. At the moment, the Code pane is empty as no code has yet been created for Owl.

In Scratch, code takes the form of **blocks** which are contained in separate coloured **palettes** according to their purpose.

Note: The blocks in each palette are listed in the *Scratch quick reference*.

Sprite speech

Select the purple Looks palette using the Looks button shown in Figure 1.4.

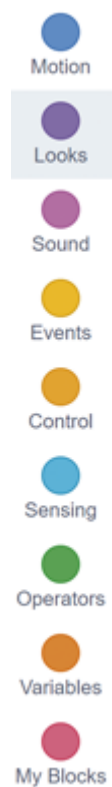


Figure 1.4

The Looks blocks appear in the palette area to the right of the buttons. You can also scroll through all blocks using the scroll bar on the right-hand side of the palette area. These blocks control the appearance of sprites (or of the stage, if that were selected instead of a sprite). This includes generating speech and thought bubbles.

Find the block shown in Figure 1.5.



Figure 1.5

When discussing Scratch blocks, it would be inconvenient to use a graphic every time we want to refer to a block, so we need textual representations. The textual representation of this block is `say[]for[]seconds`.

Now click on the `say[]for[]seconds` block, drag it from the palette into the code area on the right and drop it by letting go of the mouse button. You have copied this block from the palette into the code area, as shown in Figure 1.6. This copied block is now available in the code area for use in building a program.

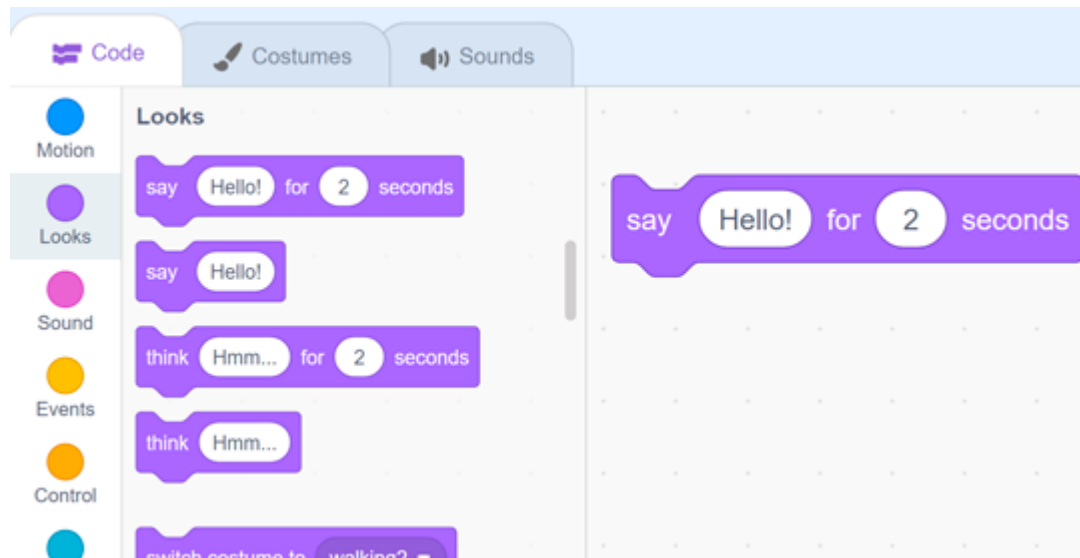


Figure 1.6

Clicking and dragging from a palette is how a block is made available for use in creating a program. A palette can be thought of as a sort of block-producing 'factory', a block in the palette being a template used to create blocks for a program.

You can see the effect this block has on the sprite by clicking on the block in the code area. (Anywhere on the block will do, except the white recessed areas which we discuss below.) Do this now, a few times, noting the effect on the sprite on the stage.

Each time you click on the block, a speech bubble emerges from Owl (to be precise, from Owl's graphic), lasting 2 seconds before disappearing. In the bubble is the text in the first input box, that is, the leftmost white recessed area of the block. Scratch supplies a default of 'Hello!'. (In TM111, when discussing text that is involved in a program we distinguish it from the surrounding text with single quote marks. The quotes are not part of the program's text.)

The programmer can either accept this default text or replace it with text of their own, as you will in this case. We refer to whatever is entered into an input box as an **input value**, or just an **input** or a **value** if the context is clear.

Click inside the first input box and type to change this input value to 'Welcome to TM111!'. (Remember not to type the quote marks.) The length of time for which the speech bubble is

displayed is specified by the value in the second input box. The default is 2 seconds. Leave this as it is, so your block is as shown in Figure 1.7.



Figure 1.7

Click on the block and Owl will 'speak' your text, in a speech bubble, for 2 seconds.

Clicking on a block results in the block being **run** or **executed**. That is, the instruction it represents is carried out. In this case, the instruction causes some text to be displayed in a speech bubble. We say that the text is *output* to the user.

Try different inputs, running your block each time by clicking on it. Finish by reverting back to the block in Figure 1.7.

Note: In our textual representation, [] indicates that a block has an input box into which a value may be entered. When referring to a block that has a specific input value, that value may be written into the textual representation – for example, [Welcome to TM111!]_{for}[2]_{seconds}.

(b)

Sprite sounds

Next you will make Owl make a sound. Blocks concerned with sound are in the Sound palette. Select this now and drag a `start_sound[ ]` block into a clear space in the code area, as shown in Figure 1.8. (In our textual representation, spaces in a block name are represented as underscores.)

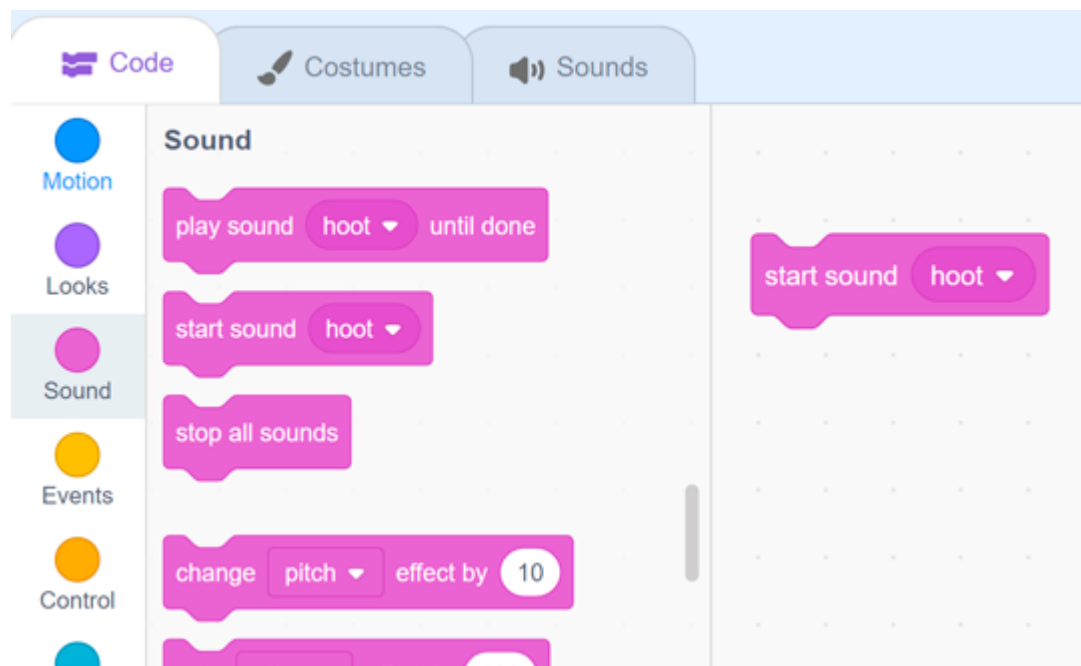


Figure 1.8

The default sound named 'hoot' fits the specification. (Other sounds could be chosen from the block's drop-down menu if they were available in the project.) Click on the block and you will hear Owl hoot.

Note: Video 1.4, which you will watch later, covers making sounds available in a project.

Remember that the program specification is for Owl to *first* welcome the user to TM111 and *then* hoot. Computers follow instructions in the order they are listed, and so you need to put the two blocks together in the required order. To do this, drag the `start_sound[ ]` block and attach it to the bottom of the `say[ ] for [ ] seconds` block, as shown in Figure 1.9.

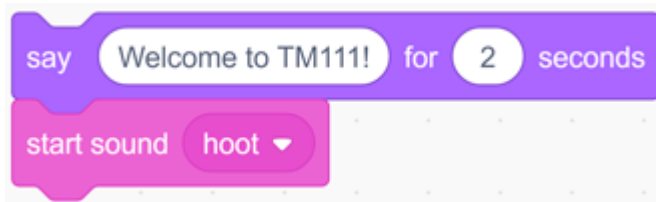


Figure 1.9

This illustrates a general Scratch feature: blocks slot together, rather like jigsaw pieces, to form larger pieces of code. However, you will see that not all blocks fit together. When moving a block, a shadow shows where it will slot together with another block. In Scratch, if two blocks don't fit, then there is something wrong with the code you are trying to create. Scratch only allows you to create well-formed code that it can run. (Whether the code does what you want it to do is another matter.)

Two or more blocks joined together in this way are known as a **stack**. When a stack **runs** or **executes**, the blocks it contains are run one by one, starting from the top of the stack. You can run a stack by clicking on any part of it (except an input box) and it will still always start running from the top. Try that now. Owl welcomes the user to TM111 and then hoots.

(c)

Events

You have seen that when you, as the programmer, click on a stack, it runs. But how does a *user*, who we have to remember may not even see the code, make it run?

You saw in Activity 1.1 that there are various ways for a user to make a program do something. The user may:

- click on the green flag icon
- click on the image of a sprite
- press a keyboard key.

Above the stage is a green flag. Clicking on this flag simulates a user clicking on the green flag icon to start the program (as you did yourself in Activity 1.1). If you click on this flag now, you will see that nothing happens.

Similarly, nothing happens if you click on Owl, that is, on the owl graphic on the stage, or if you press any of your keyboard keys.

That is because creating a stack or just a single block in the code area does not make it a part of a program. To do that, you need to add another block that tells Scratch *when* to run the stack as part of the program. No such block is currently present, therefore your stack does not run in response to user actions – in contrast to programmer actions, where clicking the stack causes it to execute.

Scratch runs a stack as part of a program only in response to a specific **event** or occurrence that the stack is associated with, such as the user actions listed above.

To associate a stack with an event requires the addition of a block from the Events palette, so select that palette now.

Most of the Events blocks have a curved top edge. Such a curved block is known as a **hat block** and it corresponds to the particular event indicated by the name of the block. It must be positioned at the top of a stack. (No other block can be positioned above it, hence its shape.) Scratch then runs the stack whenever the event corresponding to its hat block occurs.

Green flag events

The block needed here, to specify that your stack should run when the user clicks the green flag, is the block shown in Figure 1.10, which we will refer to as a `when_green_flag_clicked` block.



Figure 1.10

Any stack headed with this block runs whenever the user clicks on the green flag icon. Think of it as 'when the green flag is clicked, run this stack'.

Drag a `when_green_flag_clicked` block into the code area and position it at the top of the stack, as shown in Figure 1.11.

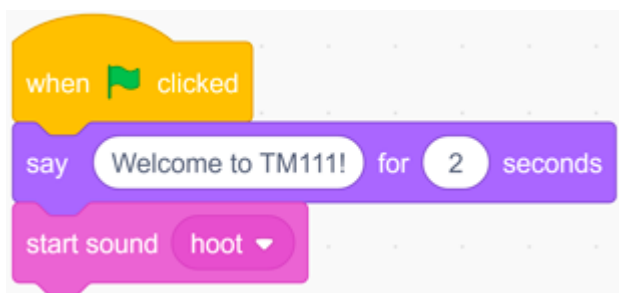


Figure 1.11

Now you have formed a stack of three blocks.

Scripts

A stack that includes a hat block is called a **script**: it is a component of a Scratch program that runs as part of the overall program, in response to a particular event. We refer to a script topped with a `when_green_flag_clicked` block as a `when_green_flag_clicked` script.

Here your program contains just one script; later you will see that a program may contain many scripts. A program may in fact contain many scripts with the same hat block, and indeed it is common to have programs containing several `when_green_flag_clicked` scripts: several different things may all need to happen at once when the user clicks the green flag icon. But for now we will just concentrate on one thing happening at a time.

Now when you click the green flag icon above the stage you will see your program in action: the script is run and the blocks are executed in order from top to bottom.

Try this now. Then run the program in full-screen mode (click on the large screen icon shown in Figure 1.12, at the top right of the Scratch window) to see the program as a user might want to do, without the code being visible.



Figure 1.12

Running a script

Revert back to the programming environment by clicking on the small screen icon at the top right of the screen.

Notice that you can run your script by clicking anywhere on it: try it. Blocks, stacks and scripts can all be run by the programmer clicking on them, but only scripts can be run when the user runs the program.

Notice also that whilst a script is running, it appears with a yellow highlight around it in the code area. This can be useful when you have a program containing several scripts so that you can see which are running at any point.

(d)

Sequencing

Suppose the program specification was for Owl to hoot *before* welcoming the user. The order of the sequence of blocks in the script would need to be changed, with the `start_sound[]` block positioned *above* the `say[]for[]seconds` block.

To try this out, click on the `start_sound[ ]` block currently at the bottom of the script and drag it directly below the `when_green_flag_clicked` block and you will see that it snaps into place as shown in Figure 1.13.

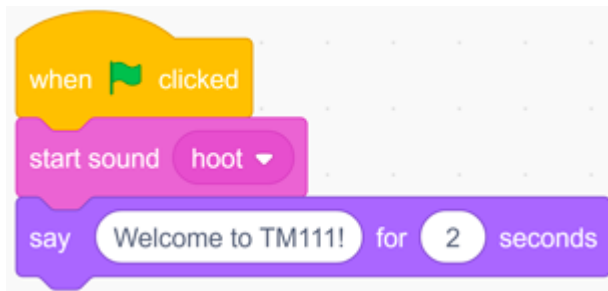


Figure 1.13

Now when you run the script, either by clicking on it or by clicking the green flag above the stage, you should see that Owl hoots and then welcomes the user. (In fact, the sound overlaps the welcome; we will discuss this shortly.)

Once you've tried this, return your script to its original state as in Figure 1.11.

(e)

For the next step in building your program, refer back to the specification at the beginning of this activity. You need to add a block so that, after hooting, the sprite 'speaks' the text 'I am the Scratch owl' – let's say for 2 seconds again. Another `say[ ]for[ ]seconds` block is needed.

The following 'Over to you' is part of an activity where you are asked to do something more independently, using what you have learned so far. A possible solution is given below the activity, but you should not look at it before you have attempted to arrive at your own solution.

Over to you

Drag in and add the appropriate block. Run your script and check that it does what is required.

You should now have a program that **implements** or **meets** the specification.

This would be a good point at which to save your program.

Solution

The block you need to add is shown in Figure 1.14, positioned at the end of the script so that it is run after the `start_sound[ ]` block.

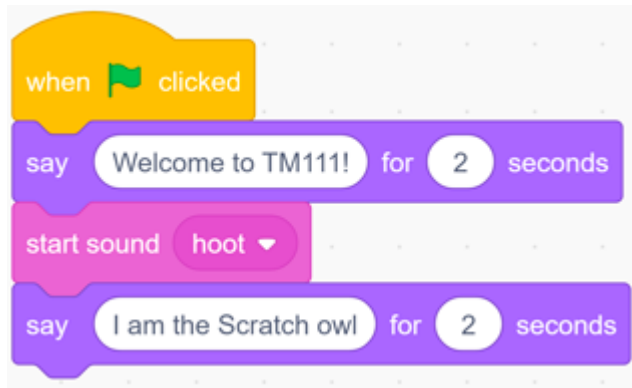


Figure 1.14

Note: You should feel free to change the display time in any `say[]for[]seconds` block to your own preference; for example, if you have difficulty seeing the screen, you may want a longer display.

(f)

When your program runs, executing the `start_sound[]` block starts the sound file called 'hoot' playing. The sound lasts a little while, and whilst it is playing your program continues to execute, moving on to the next block. Hence the sound overlaps with the display of the second speech bubble; in fact, they appear almost simultaneous as the execution of the final block happens so quickly after the execution of the previous block.

When running a stack, Scratch completes each block's execution before moving on to the next. This is true even of the `start_sound[]` block: the block's role is simply to *start* the sound running; once this is done, the sound continues as subsequent blocks are executed.

Over to you

Let's suppose we want the entire sound to complete before the second speech bubble is displayed; can you work out what alternative Sound block you should use?

Glance through the Sound blocks and try to identify a likely candidate from the block names. Simply scanning the names and experimenting with the blocks is a very good strategy when seeking a block to do a particular job. However, Scratch does offer tutorials via the Tutorials button, as shown in Figure 1.15. These allow you to explore different Scratch concepts, although be aware they do not provide direct help with the blocks. If you wish to explore the tutorials, you can do so now. There is a search function to narrow the tutorials to a particular subject, such as sound.

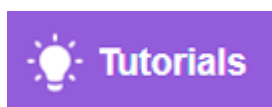


Figure 1.15

If you have accessed the tutorials, close the guidance by clicking on the back button at the top left corner, as shown in Figure 1.16.



Figure 1.16

Once you have found the block, exchange it for the `start_sound[ ]` block in your script. To do this:

- separate the blocks in the script by dragging them apart
- drag away the `start_sound[ ]` block to a clear space in the code area
- then reassemble your script including your replacement block.

Run your new script to see the effect.

Solution

The block `play_sound[ ]until_done` looks like a good bet, simply from its name. That is, the execution of the block is only complete when the sound is finished; in effect, the script's execution is paused whilst the sound is played. So, the revised script should be as shown in Figure 1.17.

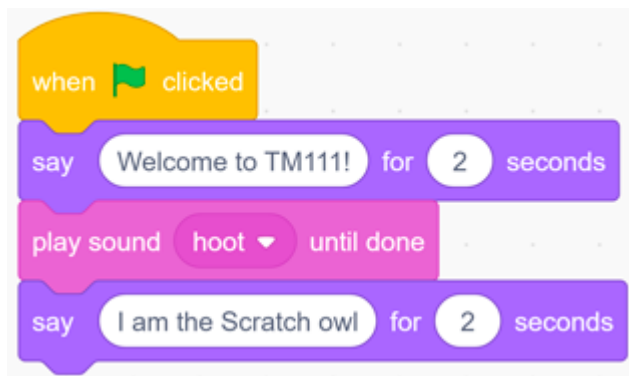


Figure 1.17

You should be able to notice (though the effect is quite subtle) that with this script the sound finishes before the second speech bubble is displayed.

(g)

There is no harm in having an unwanted `start_sound[ ]` block sitting in the code area. With no hat block, it is not part of the program, and so it has no effect when the program is run. However, it is sensible to reduce clutter in the code area by removing unwanted blocks or stacks. To do this, you can either:

- right-click on the block or stack and select 'Delete Block', or
- drag it left into the palette area where it will disappear.

(h)

Finally, experiment with manipulating your script in the code area. Notice in particular that:

- you can move it entirely by clicking and dragging on its hat block
- you can separate the blocks by clicking and dragging
- to remove a block from a stack you can:
 - (if it is the bottom block) simply move it away
 - (if it is higher up in the stack) move away the blocks beneath it, move the unwanted block away, and then reattach the bottom blocks to the stack.
- you can create a duplicate copy of a single block or of a stack of blocks by right-clicking and selecting 'Duplicate', then clicking wherever you want to position the duplicate block. If you right-click on a block within a stack and select 'Duplicate', then a duplicate is created of that block and all blocks beneath it.

Note: Video 1.2, which you will see shortly, reviews manipulating scripts.

Save your completed program in case you want to refer back to it.

A completed program is provided in `Activity_1.3_completed.sb3`.

Note: Completed programs are provided as an extra resource in case you had problems completing an activity. You may not need to look at them.

Further discussion

Here are some further notes about two of the concepts that arose in the preceding activity.

Working with blocks

Refer to the *Scratch quick reference* if you need help finding a block in a palette.

Default project

The default Scratch project is the basis for new projects in Scratch. You can obtain it either by launching Scratch afresh or by selecting 'New' from the 'File' menu. If you ever wish to try out programming ideas by yourself, then this is a good starting point. If you already have a project open, opening another project causes the previous one to close. Make sure you save the previous project, if appropriate, as it is not saved automatically.

1.1.2 Interacting with the user

Activity 1.4 (exploratory)



Watch Video 1.2, which reviews manipulating blocks and scripts, including removing, deleting and duplicating blocks. It also introduces some features of the Scratch programming environment that make working with blocks and scripts easier.

In the next activity, you will see how a program can obtain and use data from the user. You will extend the program from Activity 1.3 so that Owl asks the user for their name and then greets them using that name.

Activity 1.5 (exploratory)

Open the project `Activity_1.5.sb3`. Recall that to open a project from the Scratch programming environment, from the 'File' menu you select 'Load from your computer' and then navigate to the required project file.

This project contains the completed program from Activity 1.3.

Note: We ask you to begin certain activities with our provided program rather than your own solution to a previous activity, so that the scripts you work with are identical to ours and hence you can more easily follow our discussions.

Save it in the folder `Part 1` as `Activity_1.5_sol.sb3`. Now you will be working with a copy of the original project. By doing this, you preserve the original project `Activity_1.5.sb3` and can return to it if necessary. As you are working, remember to save your project from time to time.

Ensure Owl is selected.

(a)

User input

In this activity, you are going to extend the program so that when the green flag is clicked, Owl welcomes the user, hoots, introduces itself as the Scratch owl, and then:

- asks the user for their name, as shown in Figure 1.18
- greets the user by name, as shown in Figure 1.19, where 'Ken' is the name entered by the user in this example.



Figure 1.18



Figure 1.19

So, the program must get input from the user and use that input to form output specific to the user.

The speech bubble in Figure 1.18 looks similar to those you have seen before in Scratch, but speech bubbles must be created differently when a response is expected from the user. A block from the Sensing palette is required in this case. Select this palette now. Its blocks are concerned with enabling Scratch to access certain kinds of data. (Think of this as Scratch 'sensing' the world around it.)

Drag an `ask[ ]and_wait` block (shown in Figure 1.20) into the code area.

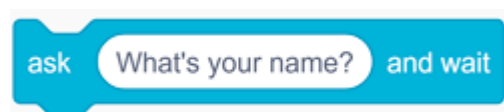


Figure 1.20

Note that its default input value is 'What's your name?'.

Click on the block and you will see its effect on Owl: a speech bubble appears containing the text 'What's your name?'. This is the same as with the `say[ ]for[ ]seconds` block, but in this case an input box also appears at the bottom of the stage, awaiting input from the user. The speech bubble remains until the user responds. When a stack containing an `ask[ ]and_wait` block is run, the execution of the stack is paused until the user responds to the request for input. (That's what the 'wait' part refers to.)

Enter your name in the input box at the bottom of the stage and press Enter on your keyboard, or click the tick at the right of the input box. The speech bubble disappears.

Change the input value of the ask[]and_wait block to 'Who are you?'. See the difference when you run the block.

Attach this block to the bottom of your script as shown in Figure 1.21 and run it.

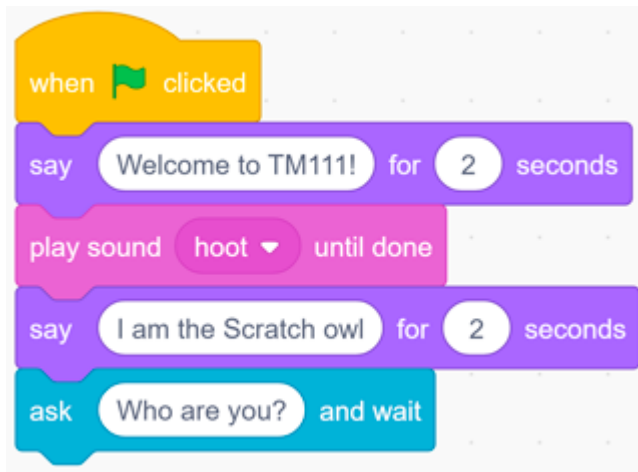


Figure 1.21

Note: Take care when using the blocks say[]for[]seconds and ask[]and_wait. The former is used when a sprite should simply 'say' or 'tell' the user something. The latter is used when data is required from the user.

(b)

Red stop icon

Both the programming environment and the full-screen mode have a red stop icon. Clicking on this icon stops all scripts that are running. As you saw with the program Octopus Hunt, some programs are *designed* only to stop when the red stop icon is clicked.

The icon within the programming environment can also be a useful tool for the programmer – for example, when you spot a problem with a script and want to work on it straight away, rather than wait for the program to end. Run your script here to the point at which Owl asks who you are. Click on the red stop icon and see that the script stops executing (it is no longer highlighted yellow) without you entering anything.

An executing script can also be stopped by the programmer clicking on it. Try this too.

(c)

Using user input

You now know how to enable a user to input data, but how can a program *use* that data? The key is the block answer. This is also a Sensing block; locate it and drag it into the code area as shown in

Figure 1.22.

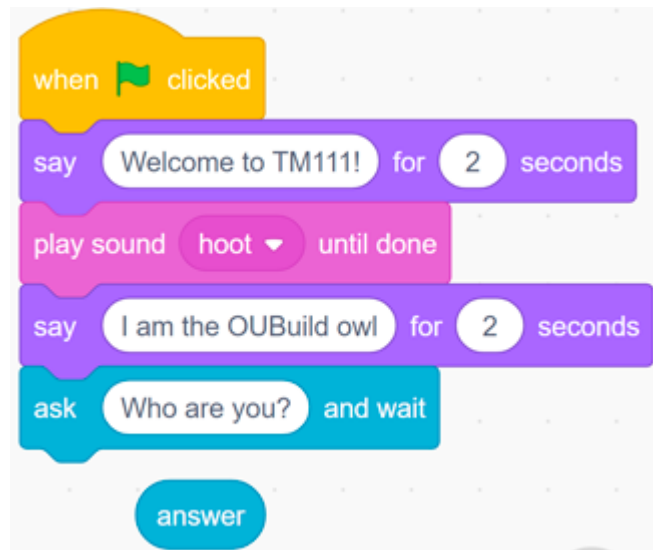


Figure 1.22

The 'oval' shape of this block indicates its specific role as a **reporter block**. Such blocks **report** or *provide* data when they are run. Here, answer reports the text that was entered by the user. We also say that **answer's value** is the text entered by the user, and that answer *gives* its value when run. This means that answer can be used subsequently in the script to give the user's name.

Run the script and enter your name when prompted. Click on answer and you will see a bubble displaying its value – that is, the name you entered previously.

There is a tick box next to answer in the Sensing palette: tick it as shown in Figure 1.23.

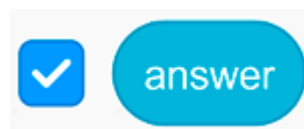


Figure 1.23

A small icon representing answer and its value appears on the stage at the top left: this is a **watcher** for answer. It displays the value of answer not only to the programmer creating a program but also to the user as they are running it. You can untick the box to remove the watcher from the stage.

You cannot simply attach an oval block to the top or bottom of another block: try it and you will see that answer just won't fit (no highlight appears). Instead, as you will see, answer and reporter blocks generally are designed to be inserted into the input boxes of other blocks. When a block which has a reporter block in an input box is run, the value of the reporter block is used as the actual input value.

Here, you can use the answer block as input to a say[]for[]seconds block, so that Owl will say your name. Select the Looks palette and drag a say[]for[]seconds block into a clear space in the code area.

Drag the answer block over its first input box (you will see a white highlight indicating that the block fits) and drop it into position as shown in Figure 1.24.

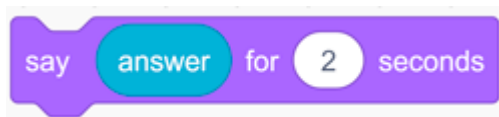


Figure 1.24

Now run this combined block by clicking on it. You will see that Owl 'says' your name in a speech bubble, such as in Figure 1.25.



Figure 1.25

When your say[]for[]seconds block is run, it uses whatever is the value of answer as its input value. The user could enter any text in response to Owl's question – any combination of keyboard characters, such as 'You talking to me?'. Scratch doesn't 'know' what constitutes an acceptable name. The user could input a meaningless sequence of characters such as '#%&! ' or even a number such as 222 and the output would simply be whatever was input, such as in Figure 1.26.



Figure 1.26

(d)

Joining strings

Recall that what we want is for Owl not only to display the user's name but to *greet* the user using that name, as shown in Figure 1.19.

Think about how you might go about achieving this. When the program runs, the user may enter any name, or indeed any text they like, so the greeting must somehow be formed whilst the program is running.

What is needed is to join 'Good to meet you ' with the value of `answer` and use the joined text as the input to the `say[]for[]seconds` block.

There is a block that does precisely this job of joining. It is in the Operators palette, whose blocks are all generally concerned with doing things to data (performing operations on data, in other words). The block needed here is the `join[][]` block, as shown in Figure 1.27.



Figure 1.27

Drag the `join[][]` block into a clear space in your code area. This block forms a function known as **concatenating**, that is, combining or joining together two *strings* into one.

A **string** is simply a programming term for a sequence of keyboard characters, which we have until now referred to simply as *text*. These characters include digits (1, 2, 3, ...), alphabetic letters ('a', 'A', 'b', 'B', ...) and other keyboard characters ('&', '\$', ...), including the space character corresponding to the spacebar on your keyboard, which we write as ' '. Examples of strings are 'Welcome to TM111!' and '£34'. A string may be empty, containing no characters. A string may contain (or even consist entirely of) one or more space characters.

(Remember that quote marks are used in our written discussions, as here, to distinguish strings, but the quotes should not be entered as input nor produced as output. We do not use quote marks around numbers since they can be distinguished sufficiently well anyway.)

Data input by the user is reported by `answer` as a string.

The oval shape of the `join[][]` block indicates that it is a reporter block, used to provide data in a program. The data it provides is the string formed by *appending* the second string to the first string – that is, adding the second string to the end of the first.

You can see that, by default, the block would join the strings 'apple ' and 'banana', to create the string 'apple banana'. Notice there is a space at the end of 'apple ', otherwise the result would be 'applebanana'.

With the block still unattached to your script, experiment by entering different values into its two input boxes and clicking on it to run it. You can 'see' the joined string reported by `join[ ] [ ]` in a small bubble.

The `join[ ] [ ]` block appends its second input value to its first input value even if the values are numerical. Try inputting 12 and 13, for example (as in Figure 1.28).



Figure 1.28

In this context, Scratch treats numbers as strings. Whether Scratch treats numerical data as a number or a string depends on the context, as you will see.

As well as joining together strings that the programmer enters directly into its input boxes, the `join[ ] [ ]` block also joins strings reported by reporter blocks, such as `answer`, when such blocks are used as input. 'Good to meet you ' is a string, and so is the value of `answer`, that is, the name entered by the user. So try what is shown in Figure 1.29, making sure to include the space at the end of 'Good to meet you '.

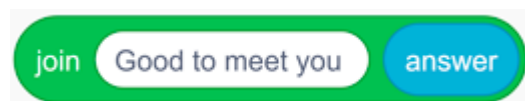


Figure 1.29

When this block is run, first the value of `answer` is found and then that value is used as input to the `join[ ] [ ]` block.

Over to you

Now combine the blocks shown in Figure 1.30 to get your script to conclude with Owl greeting the user by name.

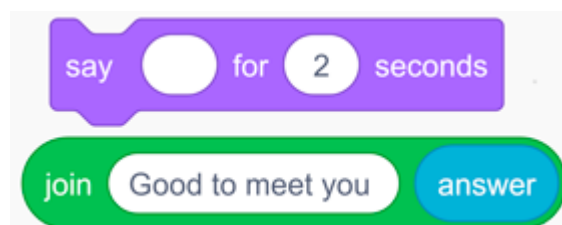


Figure 1.30

Run your script several times, entering different text for the user input. What happens if you don't enter anything but simply click on the tick?

Solution

You need to use the `join[ ][ ]` block as input to the `say[ ]for[ ]seconds` block, as shown in Figure 1.31.

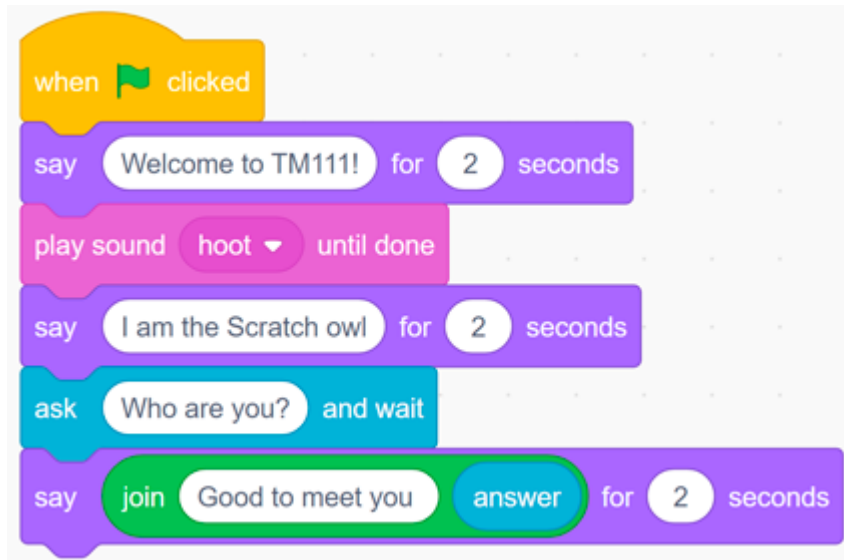


Figure 1.31

If you don't enter anything in response to the request for user input, but simply click the tick, then `answer` appears to have no value, and nothing is displayed in place of the user name when Owl speaks. In fact, `answer`'s value in this case is an **empty string** (`' '`) – that is, a string containing no characters.

Remember to save your completed program.

A completed program is provided in `Activity_1.5_completed.sb3`.

Further discussion

Block categories

You have now worked with blocks of three distinct shapes. These correspond to the three categories of block in Scratch:

- **Stacking blocks** have a rectangular shape and attach to the top or bottom of other stacking blocks to form a stack. Figure 1.32 shows an example. Stacking blocks instruct Scratch to *do* something.



Figure 1.32

(You will also meet stacking blocks with a slightly different rectangular shape, with 'jaws' between which other blocks can be positioned, such as in Figure 1.33.)



Figure 1.33

- **Reporter blocks** have an oval shape (or, as you will see later, an elongated hexagonal shape in some cases) and slot into input boxes of stacking blocks. Figure 1.34 shows an example. Reporter blocks can be used to provide programs with data.



Figure 1.34

(You will meet the different concept of a *variable* in Section 1.3. Variables are also oval in shape and provide programs with data.)

- **Hat blocks** have a curved top and attach to the top of a stack to form a script. Figure 1.35 shows an example. A hat block instructs Scratch to run the stack below it in response to a specific event.



Figure 1.35

Nesting

Certain blocks fit *inside* other blocks. This is a process called **nesting**, and a block formed by combining blocks in this way is referred to as a **nested block**.

Consider the block from Activity 1.5 shown in Figure 1.36.

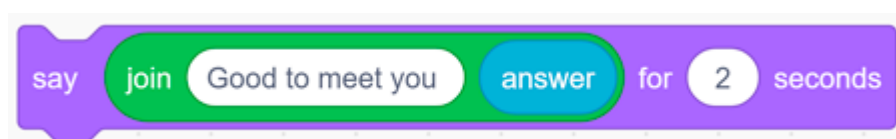


Figure 1.36

You saw that when it was run as part of your script, it resulted in Owl greeting the user by name via a speech bubble such as in Figure 1.19. Hopefully you found it reasonably intuitive to construct this block in

order to accomplish the task. However, it is worth considering in more detail how this block works as it illustrates an important general principle.

Scratch runs nested blocks starting from the innermost block and working outwards to the outermost block. So, in the example above, the order is as follows (illustrated in Figure 1.37):

1. `answer` reports the user's name
2. the name is joined to 'Good to meet you '
3. the `say[]for[]seconds` block displays the resulting string.

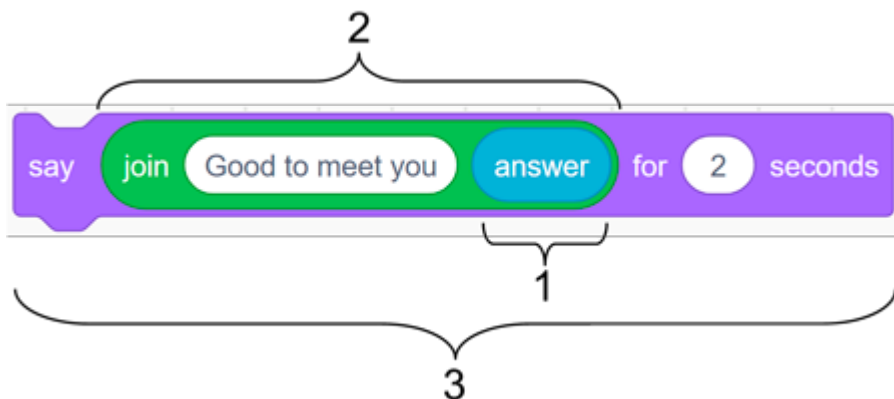


Figure 1.37

The nesting involved determines what happens when a nested block is run. The principle applies however many levels of nesting there are. The most deeply nested blocks are run first. Then the blocks in the next level out are run using the values reported by blocks nested within them. The same principle is applied again at the next level, and so on through the levels until finally the block at the outermost level is run.

The next activity is for self-assessment, so you aren't expected to create any code – though if you are stuck, you may find it helpful to do so.

Activity 1.6 (self-assessment)

Analyse the nesting involved in the stack shown in Figure 1.38 and describe what output would result if the user input 5.

Note: When we're not creating a program, we will often use examples that are simply stacks rather than complete scripts, for simplicity and brevity.



Figure 1.38

1.1.3 Conclusion

This section concludes with a programming exercise. A programming exercise is different from an exploratory activity in that you are expected to work more independently – the former typically gives less detailed teaching guidance than the latter – putting into practice and sometimes extending ideas that you have learned about. Programming exercises may sometimes introduce new blocks or new ways of using blocks. It is important you do them.

Activity 1.7 (exercise)

Open the project `Activity_1.7.sb3` by selecting 'File' at the top of the screen and then 'Load from your computer'. Remember to save the existing project if you want to keep it.

Note: We won't continue to remind you of this.

Save it immediately as `Activity_1.7_sol.sb3`.

Create a `when_green_flag_clicked` script that makes Owl do the following:

- ask the user 'How many times a week do you eat chocolate?'
- tell the user 'Only x times!', where x is whatever number the user entered, such as in Figure 1.39.



Figure 1.39

You will need to use two `join[][]` blocks and apply the ideas involved in nesting one stage further than previously. Make sure you include spaces to format Owl's speech correctly.

Reveal solution

In this section, you have been introduced to some fundamental programming ideas including how programs:

- follow instructions in sequence
- may respond to events
- may handle user input
- may involve nested instructions.

You have also been introduced to some important aspects of Scratch including:

- blocks, stacks and scripts
- sprite sounds and 'speech'
- joining strings.

1.2 Sprite movement

In this section, you will learn how to make sprites move about the stage and appear animated. You will also learn how to create scripts that run when the user clicks on a sprite's image or presses a keyboard key.

Along the way you will see how Scratch can be instructed to execute a stack repeatedly; this will be a brief introduction to a feature that you will learn much more about in subsequent parts.

1.2.1 Animation

Activity 1.8 (exploratory)

Open the project `Activity_1.8.sb3` and save it as `Activity_1.8_sol.sb3`.

This project contains three scripts that we have created.

(a)

Look first at the two scripts shown in Figure 1.42.

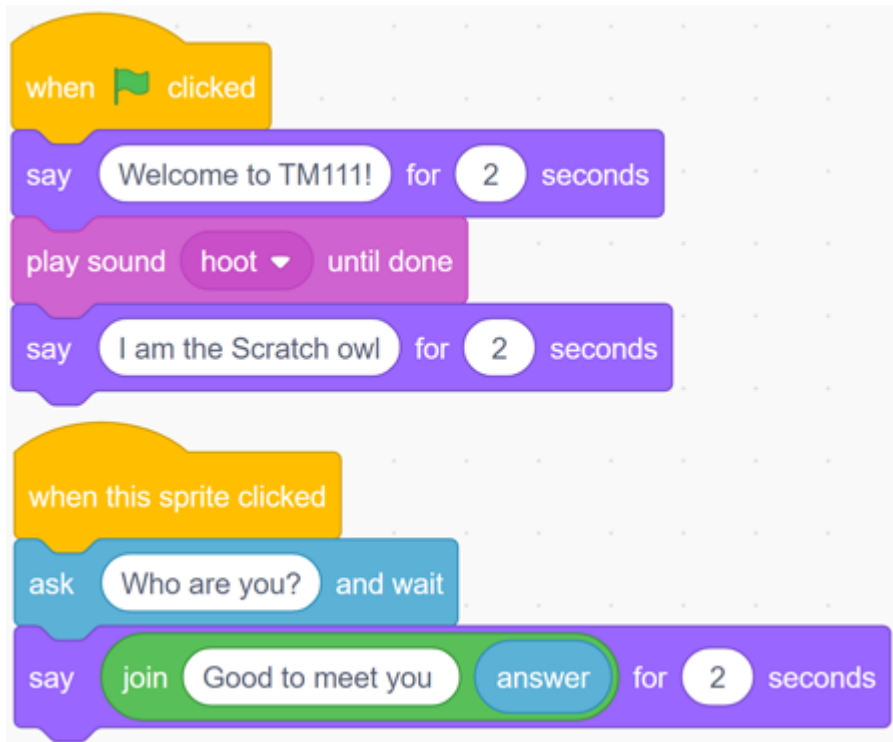


Figure 1.42

These scripts contain the same blocks from Activity 1.5 but arranged differently, under different hat blocks. One is the `when_green_flag_clicked` block that you have already met and the other is a `when_this_sprite_clicked` block which we will discuss shortly.

Click on the green flag to start the program. As you should expect from inspecting the `when_green_flag_clicked` script's code, Owl simply:

- welcomes the user to TM111
- hoots
- introduces itself.

Then click on Owl, that is, the owl graphic on the stage, and enter your name. Click on Owl again. Whenever Owl is clicked, the script headed with the `when_this_sprite_clicked` block runs and Owl:

- asks the user who they are
- greets them by name.

Again, this is what you should expect from inspecting the script.

Run your program in full-screen mode to see the full effect, clicking the green flag icon to start and then on the owl for a greeting.

Return to the programming environment.

Sprite clicked events

For a script to run in the event of a user clicking on a sprite's image on the stage, that script should:

- be created for the sprite in question (here there is only one sprite, Owl)
- be headed by a `when_this_sprite_clicked` hat block, from the Events palette.

(b)

Now turn to the other script, headed with a `when[a]key_pressed` hat block, as shown in Figure 1.43.

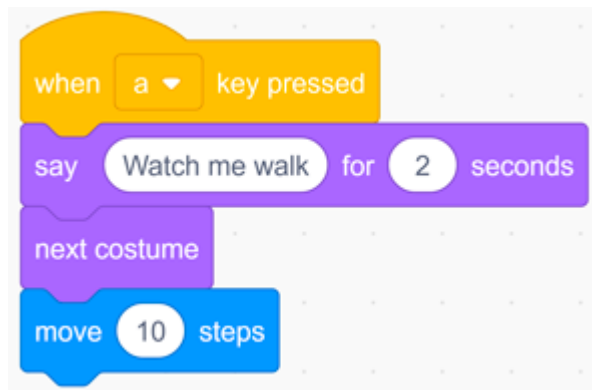


Figure 1.43

This contains some new blocks.

(i)

Press the 'a' key on your keyboard; you should see Owl speak and then 'walk' a single pace across the stage, appearing to move its legs. If you do this repeatedly, eventually Owl will encounter the right-hand edge of the stage; however, it will not move completely beyond it. You will learn about the limits of the stage after this activity.

You can drag sprites around the stage directly, using your mouse. Reposition Owl roughly in the centre again. Moving sprites around directly like this is a facility available to the programmer but not generally to the user.

Key press events

For a script to run in the event of a user pressing a keyboard key, that script must be headed by a `when[]key_pressed` hat block, from the Events palette. Look at that block (Figure 1.44) in the palette now.

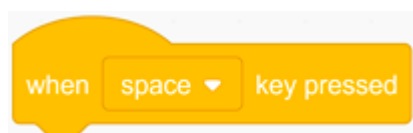


Figure 1.44

The default input value is space, corresponding to the spacebar, but other keyboard keys can be selected from the drop-down list in the input box.

Change the input to the block at the top of your script from 'a' to space. Now the script will run when you press your spacebar: try it. Revert back to 'a' before moving on.

(ii)

Directly below the hat block is a say[]for[]seconds block, which makes Owl start by speaking.

Then there are the two blocks, shown in Figure 1.45, which make Owl appear to 'walk'.

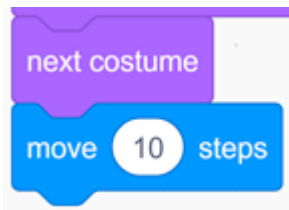


Figure 1.45

The next_costume block is from the Looks palette and the move[10]steps block is from the Motion palette.

You are going to explore each of these blocks. Place individual next_costume and move[]steps blocks in the code area, as shown in Figure 1.46. To obtain these individual blocks, you could either duplicate those at the bottom of the when[a]key_pressed script or drag blocks in from the Looks and Motion palettes.

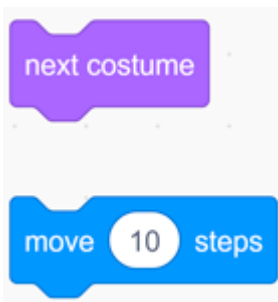


Figure 1.46

Run them (by clicking on them).

Go to the Costumes tab and look at Owl's two costumes, which illustrate the owl in two walking poses. Then return to the Code tab. The next_costume block instructs Scratch to switch to the next costume listed on the left of the Costumes tab, cycling back to the first in the list after the last has been reached. Click on the block to run it; Owl changes costume. Click again, and again. Rapid repeated execution of the next_costume block gives the impression of Owl walking on the spot.

The `move[10]steps` block moves Owl to the right. Try entering different numbers as input: negative as well as positive, decimals as well as integers (whole numbers). Positive numbers of any kind have the effect of moving Owl to the right; negative to the left. (Drag Owl back centre stage if necessary.)

Note: The input value of 10 specifies the distance to be moved, not the number of discrete movements. This block repositions Owl instantaneously; it is not shown in the process of moving to its new position.

Delete the individual blocks and return to the `when[a]key_pressed` script.

(iii)

In the `when[a]key_pressed` script, 'walking' a single pace is achieved by switching costume and moving. The animation – the apparent movement of Owl's legs – is achieved by switching costume to one showing a slightly different walking pose.

In general, the basis for sprite animation is rapid switching between costumes.

Suppose the specification for the `when[a]key_pressed` script is now as follows. When the user presses their 'a' key, Owl should:

- say 'Watch me walk'
- then 'walk' 12 paces.

Walking 12 paces *could* be achieved by constructing a stack containing 12 pairs of `next_costume` and `move[10]steps` blocks. You could (but don't) start by adding 11 more pairs of blocks, as started in Figure 1.47.

This script would be long and repetitive, so a shorter and more elegant way of achieving the same result is desirable. Fortunately, Scratch, like all comparable programming languages, has special structures that allow a particular group of instructions to be executed multiple times.

The block needed to specify the repeated execution required here is in the Control palette, which contains blocks concerned with controlling the execution of a program. It is the `repeat[ ]{ }` block shown in Figure 1.48.

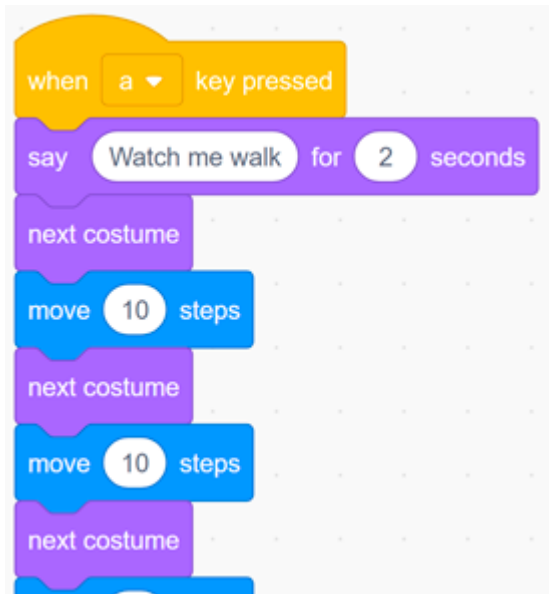


Figure 1.47

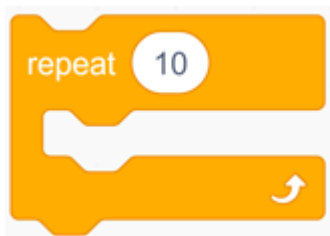


Figure 1.48

Drag it to the code area. This block is shaped with upper and lower 'jaws' which are designed to be positioned around either a single block or a stack of blocks. (The {} in our textual representation represent the block's jaws.) The input value (the default is 10) specifies how many times the block(s) within its jaws are to be run.

Drag the repeat[]{} block over the two blocks at the bottom of your script and you will see its jaws expand and a grey outline appear indicating that its jaws can be positioned around the blocks. Drop it into place so that your script looks like that in Figure 1.49.

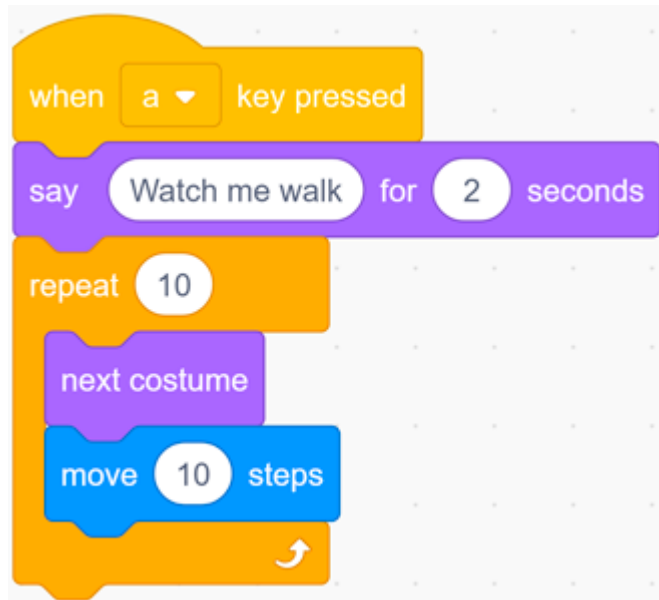


Figure 1.49

We want the stack of blocks within the jaws to be run repeatedly, 12 times. This requires you to set 12 as the input value in the repeat[]{} block's input box, as shown in Figure 1.50. Do that now.

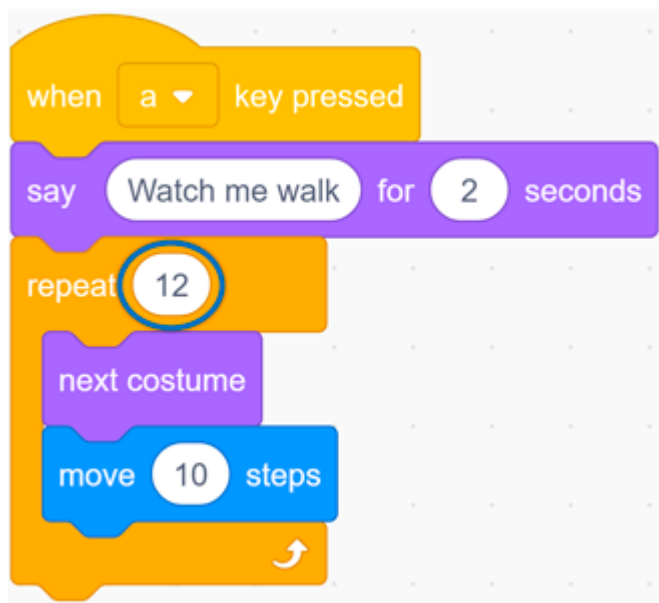


Figure 1.50

When it is run, the block repeat[12]{} instructs Scratch to run the stack of blocks between its jaws 12 times.

Starting with Owl roughly centre stage, run the script. Owl appears to walk 12 paces.

Note: You may find that this animation is not apparent as the costume changes may be made too quickly to be visible. In a later activity, we will show you how to deal with this.

(iv)

This is a more elegant and flexible solution than creating a lengthy stack of duplicate blocks as in Figure 1.47. Suppose the specification changed to require Owl to move 15 paces; how would you amend the script?

You would replace the 12 in the input box by 15, as shown in Figure 1.51. Try it.

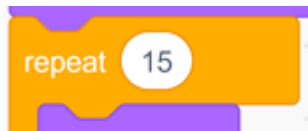


Figure 1.51

Try entering different numbers into the input box and running the script. Then reset the input to 12.

It is important to include the right blocks within the jaws. See the difference if you temporarily position the say[]for[]seconds block within the jaws, as shown in Figure 1.52.

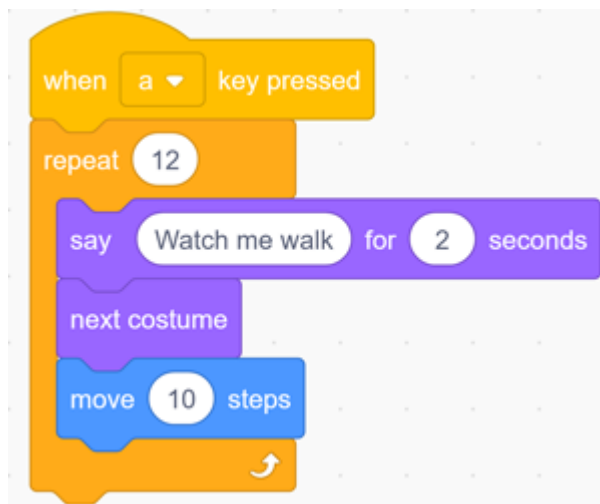


Figure 1.52

Then return the say[]for[]seconds block to its original position, as shown in Figure 1.50.

(c)

This part of your program is complete, so run your program and try it out. Start by clicking on the green flag icon. Then click on Owl and press your 'a' key a few times. Owl will greet you and then walk on each press.

Restart your program by clicking on the green flag icon and notice that Owl starts off at the place on the stage where it finished in the previous run of the program. In Activity 1.11 you will amend this so that Owl always starts from the same position whenever the program starts.

(d)

Try clicking on the green flag icon and then *immediately* press your 'a' key.

Exactly what happens depends on how quickly you can click. But you should see that as soon as you press your 'a' key, the when[a]key_pressed script begins running, even while the when_green_flag_clicked script is still running. You may see Owl tell the user to watch it walk before it introduces itself.

A completed program is provided in `Activity_1.8_completed.sb3`.

Further discussion

Events

You've now met the following three kinds of user-generated events that scripts can be created to respond to.

- The user clicks the green flag icon. This runs any script, for any sprite, headed with a when_green_flag_clicked hat block. We call such a script a **when_green_flag_clicked script**. Recall that by convention, unless the user is directed otherwise, clicking the green flag icon is the way to start a Scratch program: we describe a program as *starting with the green flag*.
- The user clicks on a sprite on the stage. This runs any script *for that sprite* headed with a when_this_sprite_clicked hat block. We call such a script a **when_this_sprite_clicked script**.
- The user presses a keyboard key. This runs any script, for any sprite, headed with a when[]key_pressed hat block with input value corresponding to that key. We call such a script a **when[]key_pressed script**.

Concurrency

It is possible to have two scripts running together: we say they run **concurrently**. In fact, Scratch only executes one block at a time. When more than one script is running, Scratch rapidly switches between the blocks in the scripts.

You will later see programs in which having multiple scripts running at once is an intentional aspect of a program.

Looping

The repeat[]{ } block is one way in which the powerful programming concept of *repetition* or **looping** is achieved in Scratch. You will learn about others in due course. A **loop** is a programming structure that enables repeated execution of a sequence of instructions.

1.2.2 Sprite position and direction

You saw that each time your program in Activity 1.8 started with the green flag, Owl began in different positions according to where it ended up the last time the program ran. Depending on the precise specification of a program, it may be necessary for certain features of a sprite, such as its position on the

stage, to be the same each time the program starts. This is part of **initialisation** – more generally, setting certain features of a program at a particular point, often the start – and is an important concept in programming. Without appropriate initialisation, programs may not work correctly.

To fix Owl at the same position each time the program starts, you need to know how the position of a sprite on the stage is determined, and that is what we turn to next. A related concept is that of a sprite's *direction*. We will discuss these ideas and you will then put them into practice in Activity 1.11.

Computer graphics describes positions on a screen in terms of horizontal coordinates (also known as *x* coordinates) and vertical coordinates (*y* coordinates). The Scratch stage is 480 units wide and 360 units tall, and the centre of the stage has an *x* coordinate of 0 and a *y* coordinate of 0. The *x* coordinates increase from left to right; the *y* coordinates increase from bottom to top. This means that, for example, the bottom right-hand corner of the stage has an *x* coordinate of 240 (that is, 240 units to the right of the centre) and a *y* coordinate of -180 (180 units down from the centre).

The coordinates of a point on the stage are sometimes written in the form (*x*, *y*). So, for example, the centre of the stage has coordinates (0, 0) and the bottom right-hand corner of the stage has coordinates (240, -180). Figure 1.53 illustrates the coordinate system of the stage, labelling the centre and corners of the stage. You can also see that, as another example, the point (-50, 50) has been labelled (50 units to the left of the centre and 50 units up from the centre).

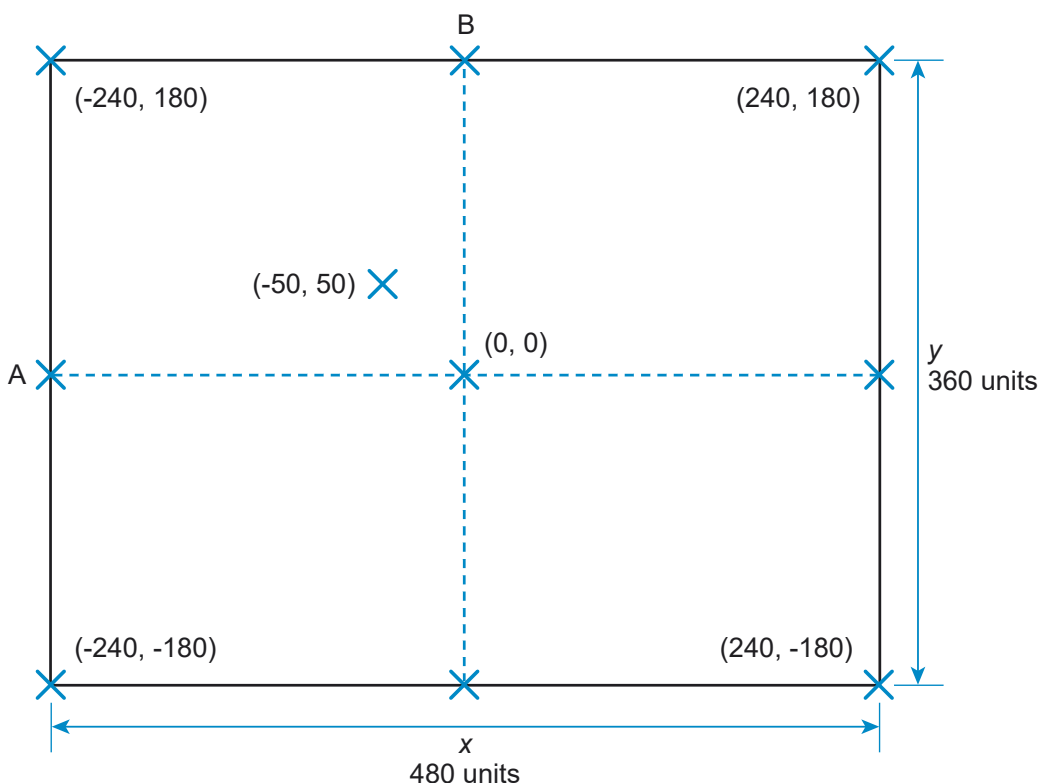


Figure 1.53

Note this carefully. A different coordinate system commonly used in computer graphics has the point (0, 0) lying at the top left of the screen, so the *y* coordinates increase as you move down the screen, which is the reverse of what happens in Scratch.

So, each point on the stage has an *x* coordinate and a *y* coordinate.

Each sprite also has an associated *x* coordinate and *y* coordinate, which determine its position on the stage. These are referred to in Scratch as the sprite's ***x* position** and ***y* position**.

How do a sprite's *x* position and *y* position relate to its movement? Well, it's much as you would expect. Movement and the coordinate system both use the same units. So, a sprite that moves 10 steps horizontally to the right, for example, will increase its *x* position by 10. And a sprite that moves 10 steps vertically upwards will increase its *y* position by 10.

A sprite's coordinates can, in fact, go a little beyond the boundaries of the stage, but you will not need to consider the details of this. As you have seen, sprites do not move entirely off the stage.

Activity 1.9 (self-assessment)

- a. What are the coordinates of the points marked A and B in Figure 1.53?
- b. If a sprite is at position (−50, 50) and its *y* position is increased by 40, at what position does the sprite end up?
- c. If a sprite is at position (−50, 50), by how much must its *x* and *y* positions be increased or decreased to move the sprite to the position (240, 0)?

As you will see shortly, the Motion palette contains several blocks concerned with a sprite's position, including the `move[ ]steps` block you have already met. It also contains several blocks concerned with a sprite's *direction*, which we introduce briefly now.

Sprite direction

At any point in time, a sprite has a particular position. It also has a particular *direction*. This determines which way the sprite will go when a `move[ ]steps` block is run. It can also determine which way the sprite's costume graphic on the stage is facing. In Scratch, direction is measured in degrees from vertically upwards, as shown in Figure 1.54.

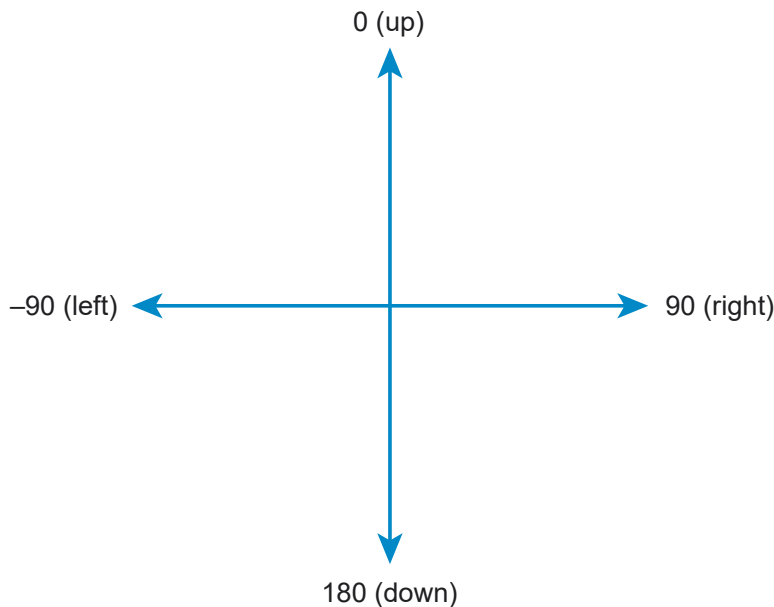


Figure 1.54

The numbers on this diagram are those used by Scratch to refer to the directions left, up, and so on. Scratch uses the range -180 to 180 to represent angles in degrees (with -180 being equivalent to 180).

Again, take note of this, as it is different from other systems used to measure direction – for example, the system commonly used in mathematics in which 0 degrees is taken to point to the right, and degrees increase anticlockwise.

Activity 1.10 (self-assessment)

If a sprite's direction is 0 degrees and a `move [ -50 ] steps` block is run, what is the effect on its x position and on its y position?

In the next activity, you will explore some blocks concerned with sprite position and direction. You will then amend the program from Activity 1.8 so that Owl's position is initialised when the program starts, and you will explore and create some other scripts involving Owl's position and direction.

Exploring blocks

Whenever you use a block for the first time, you should explore it. Exploring a block involves running it by clicking on it. We will guide you through specific ways to run some blocks where, for example, there are particular features we want to explain.

We expect that you are now working more quickly within the Scratch programming environment and needing less guidance on mechanical matters such as block manipulation. So, we will start to guide some of your investigations in the form of tables, for simplicity and brevity. The left-hand column of the table tells you what you should do and sometimes gives you questions to think about. The right-hand column gives notes on your explorations and answers to some of the questions.

Activity 1.11 (exploratory)

Open the project `Activity_1.11.sb3` and save it as `Activity_1.11_sol.sb3`.

This project contains the completed program from Activity 1.8 with the addition of a `when[b]key_pressed` script, which you will explore in due course.

(a)

Position

You'll start by investigating sprite positions. On that basis, you will then amend the `when_green_flag_clicked` script so that Owl's position is initialised when the program starts.

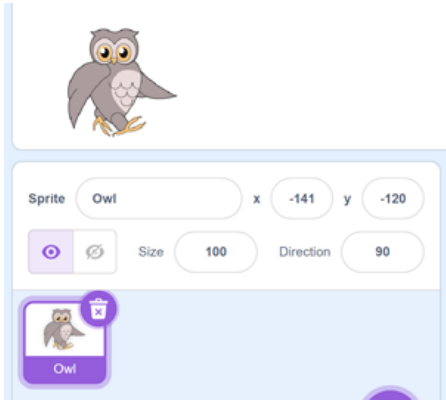
To do	To note
Drag the owl sprite to a new position on the stage. Review Owl's information pane, which will update with the new <i>x</i> and <i>y</i> positions when the sprite is dropped.	Owl's information pane displays key features of Owl, including its <i>x</i> and <i>y</i> positions, as shown in Figure 1.55.  Figure 1.55
Blocks from the Motion palette enable a program to manipulate a sprite's position. You have already met the <code>move[]steps</code> block. Select this palette now and look for the <code>go_to_x:[]y:[]</code> block shown in Figure 1.56. Don't move it into the code area quite yet. What do you notice about its default input values? 	The default input values are the current <i>x</i> and <i>y</i> positions of Owl.

Figure 1.56

Now drag the block to a clear space in the code area and explore it, entering different input values and observing the effect on Owl when it is run.

To do

Ensure, by dragging if necessary, that Owl is positioned away from the centre of the stage. Set both input values of the `go_to_x:[]y:[]` block to 0, as shown in Figure 1.57, corresponding to the centre (0, 0) of the stage. Run the block.

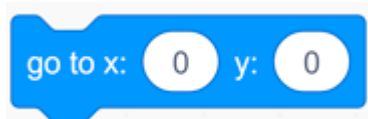


Figure 1.57

To note

When both input values are set to 0, Owl moves centre stage.

Some other Motion blocks that manipulate a sprite's position are shown in Figure 1.58.

One by one, drag each of these blocks to the code area and explore it.

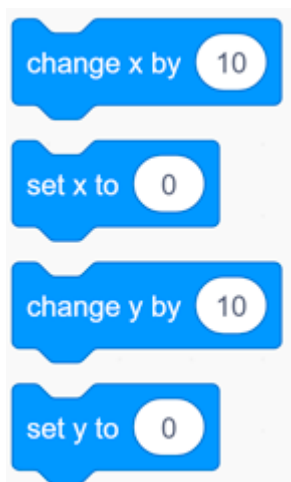
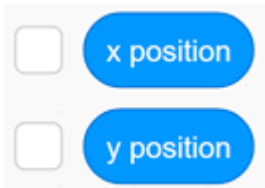


Figure 1.58

These blocks enable the x position and the y position of a sprite to be set to specified values, or to be changed by specified amounts (negative or positive).

To do	To note
Now turn to the Motion blocks shown in Figure 1.59.  Figure 1.59 One by one, drag each of these blocks to the code area and explore it, moving Owl around and clicking on the blocks to reveal their values. What type of blocks are these?	Their oval shapes indicate that they are reporter blocks, reporting the numbers that are the sprite's x and y positions.
What happens if you click the tick box next to each of them in the palette area and then move Owl round the stage?	Clicking a tick box results in a watcher for that block appearing on the stage. Just as with the watcher for answer, each of these displays the current value of the block – here, the current x or y position. As you move Owl around the stage, they reflect its changing position.
Create and run the nested block shown in Figure 1.60.  Figure 1.60 (Remember that the say[]for[]seconds block is in the Looks palette and the join[][] block is in the Operators palette.)	Owl tells the user its x position.

(b)

Now let's turn to initialising Owl's position. Specifically, let's suppose that when the green flag is clicked, Owl should:

- *go to the centre of the stage*
- welcome the user to TM111

- hoot
- introduce itself.

The part in italics is new.

Over to you

Use a `go_to_x:[ ]y:[ ]` block to amend your `when_green_flag_clicked` script to meet this revised specification.

Start your program with the green flag. Make Owl walk a few times by pressing your 'a' key. Then start the program again. You should see that Owl appears centre stage each time the program starts.

Solution

A solution is shown in Figure 1.61.

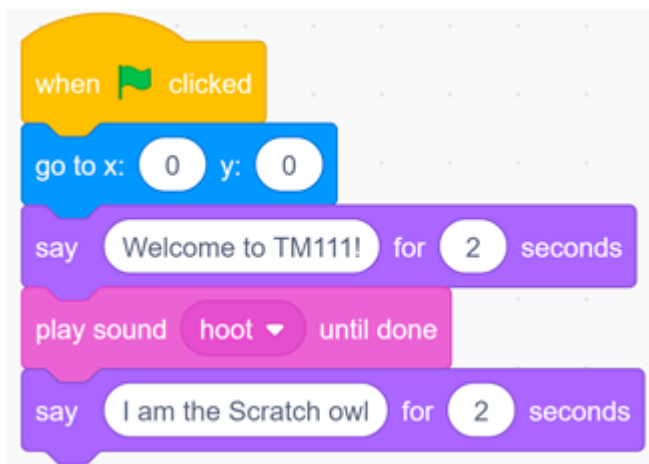


Figure 1.61

(c)

Now turn to the `when[b]key_pressed` script provided.

Run it a few times. It makes Owl speak and then appear to jump vertically up and then down.

How does it work? The 'up' part is implemented by the block `change_y_by[40]`, which increases Owl's y position by 40. The 'down' part is implemented by the block `change_y_by[-40]`, which decreases Owl's y position by 40.

The `wait[ ]seconds` block, from the Control palette, is new. Temporarily remove it from the script, as shown in Figure 1.62, and run the script to see the problem that arises without it.

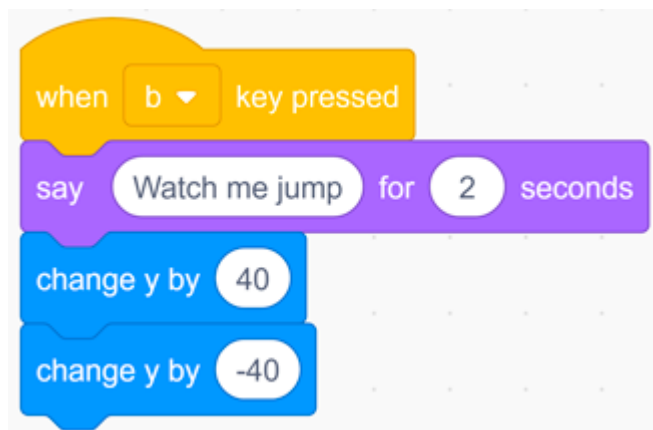


Figure 1.62

The 'jump' is not visible. The `change_y_by[ ]` blocks are executed so rapidly one after the other that the change in y position is not visible.

The `wait[ ]seconds` block instructs Scratch to pause execution of a stack for the number of seconds specified by its input value. Its inclusion in the `when[ b ]key_pressed` script implements a pause between the 'up' and 'down' parts of Owl's jump, enabling the jump to be visible.

Return the `wait[ ]seconds` block to the script.

Note: If you wish to slow down the costume changes when the `when[ a ]key_pressed` script is run, so that you can see Owl's animation more clearly, you can insert a `wait[ ]seconds` block inside the `repeat[ ]{ }` block's jaws.

(d)

Direction

By default, that is, when it is newly created, a sprite has position (0, 0) and direction 90, so it is 'pointing' horizontally to the right. This is why Owl currently heads to the right when instructed to move – by a `move[ 10 ]steps` block, for example.

The next scripts you will look at will involve changing Owl's direction, so first you'll explore some relevant blocks.

To do	To note
Click on the direction box in Owl's information pane, shown in Figure 1.63.	A sprite's direction is displayed in its information pane.

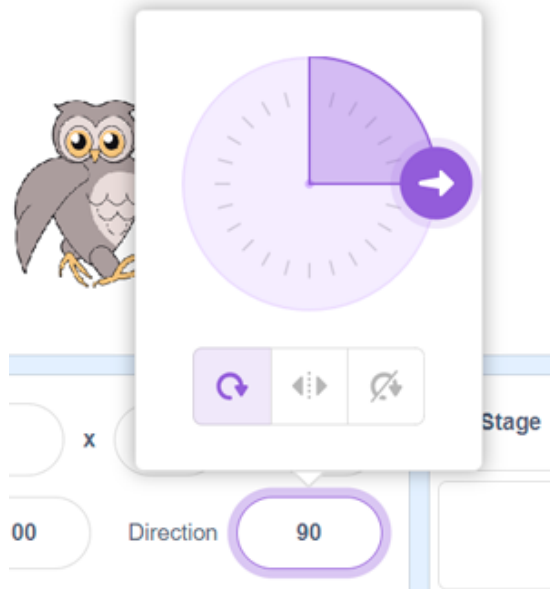


Figure 1.63

The small arrow in the circle (shown in Figure 1.64) in fact allows the programmer to set the sprite's direction directly by moving the line around the circle: try this now.



Figure 1.64

On the stage, Owl turns to point in the specified direction. (See Figure 1.54 for a reminder of how directions are represented in Scratch.)

(If the owl image doesn't turn according to the direction, you may have changed Owl's rotation style. In the direction pop-up box, the rotation style should be set as shown in Figure 1.65. You will look further at rotation styles shortly.)

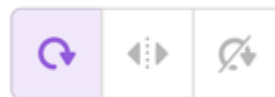


Figure 1.65

Set Owl's direction differently a few times, running the when[a]key_pressed script each time. What happens?

Owl walks in the direction you have set. The move[]steps block moves a sprite according to the sprite's current direction.

To do

Within a program, a sprite's direction can be set using various Motion blocks

To note

Clicking on the point_in_direction[] block's input box allows you to set the sprite's direction in 15-degree

– in particular, the `point_in_direction[]` block shown in Figure 1.66. Drag this block into the code area and explore it.

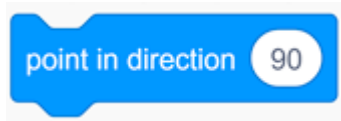


Figure 1.66

intervals, but you can enter any number directly, for example, 43.

Check what happens when you enter directions outside the range that Scratch uses (–180 to 180), such as in Figure 1.67. Note that the direction wheel also appears above the input box.

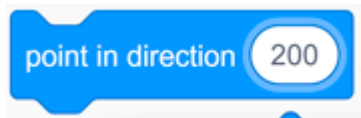


Figure 1.67

Owl's direction is set to one in the range –180 to 180 (the range Scratch uses). This is obtained by adding or subtracting multiples of 360 to the input value. Scratch does this conversion behind the scenes. For example, `point_in_direction[200]` sets Owl's direction to –160. ($200 - 360 = -160$).

Adding or subtracting multiples of 360 gives equivalent directions since 360 degrees represents a full turn. So, for example:

- –90 degrees is equivalent to 270 degrees
- 0 degrees is equivalent to 360 degrees
- –180 degrees is equivalent to 180 degrees.

All the blocks concerning direction recognise such equivalence.

Other blocks that change a sprite's direction include those in Figure 1.68.

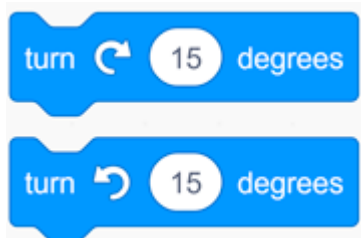
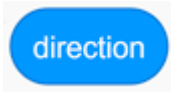


Figure 1.68

We refer to these as `turn_clockwise[]degrees` and `turn_anticlockwise[]degrees`, respectively. They change the sprite's direction by, respectively, adding or subtracting their input value. Each of them also allows a negative input value, meaning that they can in fact make a sprite turn the opposite way.

Explore each of these.

To do	To note
There is a reporter block, <code>direction</code>, as shown in Figure 1.69.  Figure 1.69 Drag it into the code area and explore it, clicking on it to reveal its value. Try clicking on the tick box next to the block in the palette.	This block reports the number that is the sprite's direction. Clicking the tick box results in a watcher for <code>direction</code> appearing on the stage, which displays the current direction.
Change Owl's direction using its information pane. What do you notice about its direction as displayed in the watcher?	The watcher shows that the sprite's direction is a decimal number that may not be an integer (whole number).

(e)

Now you should be equipped to work with a sprite's direction within a script.

Over to you

Create a script for Owl so that when the user presses their 'c' key:

- Owl asks the user what direction (between -180 and 180) it should face (use an `ask[]and_wait` block)
- Owl's direction is set to whatever the user inputs (use `answer` as input to a `point_in_direction[]` block).

Run your new script.

What happens if the user's input is not numerical, for example, 'three'? What happens if the input is a number outside the range -180 to 180 ?

Solution

A solution is shown in Figure 1.70.

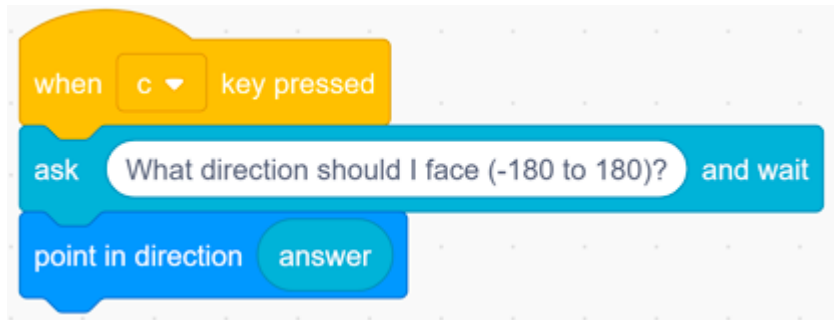


Figure 1.70

If the user's input is not numerical, for example, 'three', then Owl's direction is set to 0. This is a common Scratch feature: if non-numerical input is supplied as input to a block requiring a number, then the input is taken to be 0.

If the user's input is outside the range -180 to 180 , then it is 'converted' to an equivalent direction inside the range by adding or subtracting multiples of 360.

(f)

With a watcher for direction on the stage, start your program with the green flag and then change Owl's direction using your 'c' key. Start the program again and observe Owl's direction via the watcher.

Just as with its position before you initialised it, Owl's direction differs when the program starts according to whatever it was last set to in the previous program run.

So, now that the user can change Owl's direction, it is sensible to consider whether its direction should be set to a specific value when the program first starts – that is, whether its direction should be initialised, so that the program always starts consistently.

Over to you

Let's suppose that Owl's direction should be set to 90 (that is, facing to the right) when the program is started using the green flag, before Owl starts speaking. Amend your program to achieve this.

Solution

A solution is shown in Figure 1.71.

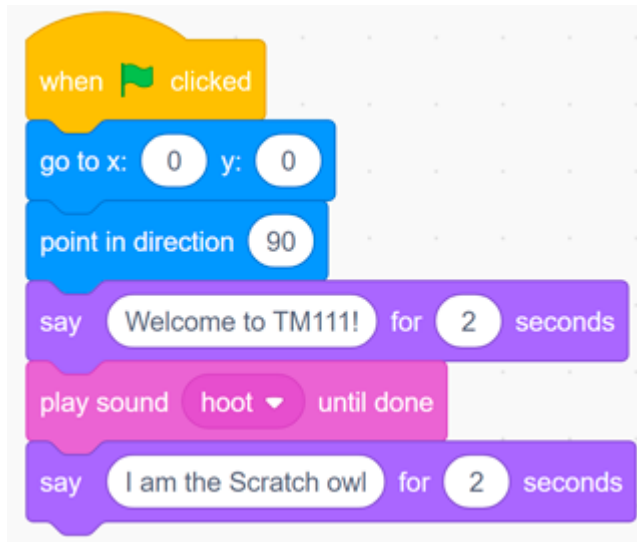


Figure 1.71

The block `point_in_direction[90]` could instead be positioned directly underneath the hat block, since no particular order is specified for the repositioning and the redirecting.

(g)

Finally, you will amend the `when[a]key_pressed` script so that when Owl reaches an edge of the stage, it doesn't stay there, but 'bounces' back within the stage.

The Motion block `if_on_edge,_bounce` shown in Figure 1.72 is the one you need. It essentially 'reflects' a sprite off an edge by changing its direction. (Its visual effect depends on the sprite's rotation style, as you will see shortly.)



Figure 1.72

Explore this block by dragging Owl to an edge of the stage and then clicking on the block.

Over to you

Position an `if_on_edge,_bounce` block within the `when[a]key_pressed` script so that immediately after Owl moves, if it has reached an edge it is bounced back onto the stage.

Run your amended script repeatedly to make Owl walk to an edge of the stage and see the effect. Change its direction and make it walk towards another edge.

Solution

A solution is shown in Figure 1.73.

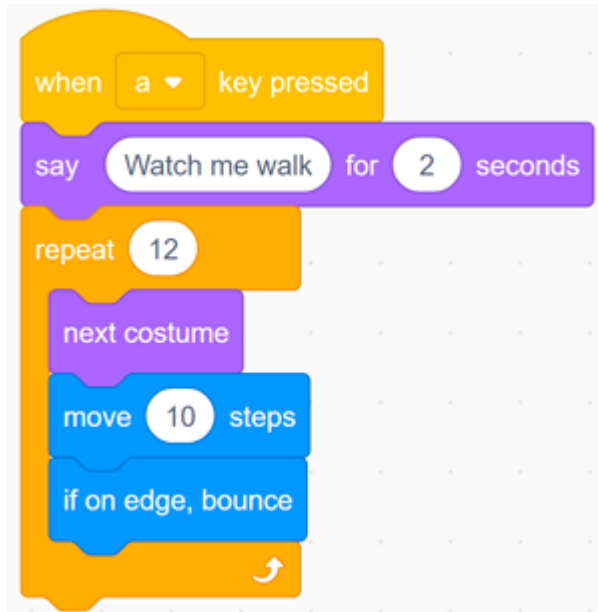


Figure 1.73

(h)

Try the full effect of your program, in full-screen mode. Start your program with the green flag. Then make Owl walk, jump and change direction by pressing your 'a', 'b' and 'c' keys.

Try pressing 'a' rapidly followed by 'b' to make Owl jump as it walks. (The jump may be hard to see mid-walk.)

Explore the effect on Owl's appearance throughout the program of setting different rotation styles in Owl's Direction information pane, which is displayed when you click on the direction, as shown in Figure 1.74. Note especially what happens when Owl 'bounces' off a stage edge. The rotation style in Figure 1.74(a) allows its costume to rotate to follow the direction. The rotation style in Figure 1.74(b) allows its costume only to be reflected vertically according to the direction. The rotation style in Figure 1.74(c) prevents its costume rotating or reflecting.



Figure 1.74

A completed program is provided in `Activity_1.11_completed.sb3`.

Further discussion

Initialisation

Depending on its specification, certain aspects of a program that may change while the program is running may need to be set every time the program starts; we say these aspects need to be *initialised*. So far, you have initialised aspects visible to the user such as a sprite's position, but later you will learn that less obvious aspects may need to be initialised as well.

You have learned about initialisation within a `when_green_flag_clicked` script, which runs when a program starts, but initialisation may be necessary within other scripts depending on what is required of them.

Activity 1.12 (self-assessment)

Suppose the specification of the `when[a]key_pressed` script were as follows.

When the user presses their 'a' key, Owl should:

- say 'Watch me walk'
- and then 'walk' 12 paces *starting from the centre of the stage*.

What change would be required to the `when[a]key_pressed` script (shown in its current state in Figure 1.73)? Don't make the change unless you are stuck; just think about what would be required.

Green flag to start

It is a Scratch convention that, unless directed otherwise, a user should initiate their interaction with a program by clicking the green flag. One reason for this is that a `when_green_flag_clicked` script may include necessary initialisation.

However, in the program in Activity 1.11 and in many others, there is nothing to force the user to click the green flag first; here they could choose to press their 'a' or 'b' key first without ever clicking the green flag, and there would be no obviously untoward effect. In more complicated programs, it is up to the programmer to direct the user how to use a program.

1.2.3 Conclusion

Activity 1.13 (exercise)

Open the project `Activity_1.13.sb3` and save it as `Activity_1.13_sol.sb3`.

In this activity, you will practise:

- exploring new blocks
- working with sprite movement.

(a)

Explore the block `pick_random[]to[]` from the Operators palette. How could you use it in combination with a `say[]for[]seconds` block to make Owl say a random number from -100 to 100 , such as in Figure 1.75?



Figure 1.75

(b)

Explore the block `glide[]secs_to_x:[]y:[]` from the Motion palette.

(c)

Create a new script for Owl so that Owl moves in some way when the user presses their spacebar. Your script should use:

- the blocks `pick_random[]to[]` and `glide[]secs_to_x:[]y:[]`
- if you wish, the `wait[]seconds` and `repeat[]{} blocks`.

You don't need to make the sprite appear animated.

Remember to:

- use the *Scratch quick reference* if necessary
- experiment – run blocks on their own or in small stacks to help you understand their effect.

Reveal solution

In this section, you have been introduced to sprite movement and animation, and to the concept of initialisation. You have also learned more about events that a program may respond to, such as a user pressing a keyboard key.

1.3 Variables

Programs often need to keep track of data while they are executing. Scratch has a number of built-in things it keeps track of, such as the user's input in response to an `ask[ ]and_wait` block, the position and direction of a sprite, and so on. These correspond to values that programs can find using reporter blocks, as you have learned.

But what if a program needs to keep track of something not built in? Counting the number of password attempts made by a user is a good example.

Programming languages provide a mechanism known as a **variable** for doing this. You can think of a variable as being a sort of named box in Scratch's memory into which a program can put an item of data, and then retrieve it again later. The reason why it is called a variable is that the **value** it holds – the stored data – may vary over time.

The programmer chooses the name of a variable. Imagine for example that we wanted a variable to store the number of password attempts. Then we can create a variable, which you can see from Figure 1.78 looks like a reporter block, giving it a suitable name – attempts, say. You will see how this is done shortly.



Figure 1.78

A variable block works in much the same way as a reporter block, as you will see. However, unlike a reporter block, the data held in a variable doesn't get stored or changed automatically; the program has to use appropriate Scratch blocks to do these things.

Thinking about our password example, when the program starts, the number of attempts is zero. Figure 1.79 shows how the variable can be set to the appropriate value.

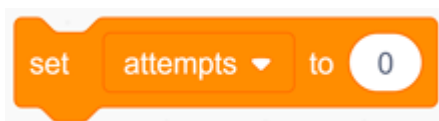


Figure 1.79

We say the `set[attempts]to[0]` block *sets* the value of the variable `attempts` to `0`. Thinking about variables as named boxes in memory, the box named `attempts` now contains the number `0`.

When the user makes an attempt, the value should increase by one. There's a block for that, shown in Figure 1.80.



Figure 1.80

The number in the attempts memory 'box', which was originally 0, is increased by one and so is now the number 1.

These are in fact the main blocks that you will use when working with variables. We hope you already get the impression that they enable working with variables to be quite intuitive. The next activity shows you how to create a variable and then gives you practice in working with these blocks.

1.3.1 Introduction to variables

Activity 1.14 (exploratory)

Open the project Activity_1.14.sb3 and save it as Activity_1.14_sol.sb3.

This project contains the completed program from Activity 1.11.

Let's suppose we now want the program to display the number of times Owl is instructed to walk, from when the program starts with the green flag. When the 'a' key is pressed, Owl should speak and then walk, just as before, but should then tell the user how many walks it has now done, such as in Figure 1.81.



Figure 1.81

To do this, the program needs to keep a count of how many times the when[a]key_pressed script runs. There is no reporter block reporting this data, so a variable is needed to hold the count. Follow the instructions below to create and use a variable for this purpose.

(a)

To do	To note
In Scratch, variables are created using the Variables palette. Select this now.	

Click on the 'Make a Variable' button, shown in Figure 1.82.



Figure 1.82

A dialog box appears, as shown in Figure 1.83.

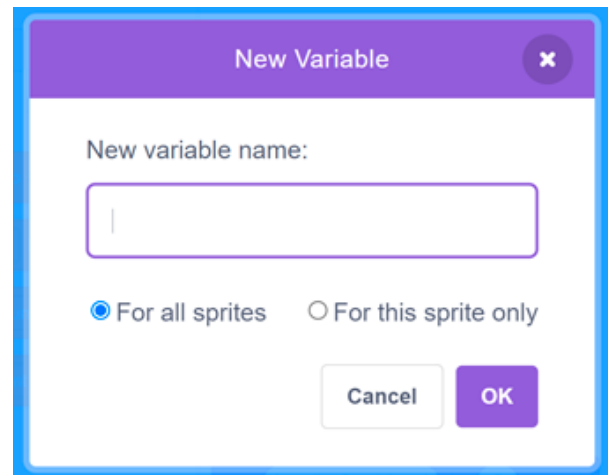


Figure 1.83

In this box enter the name `walk_count`, as shown in Figure 1.84. (Include the underscore in your variable name. Variable naming is discussed later in this section.)

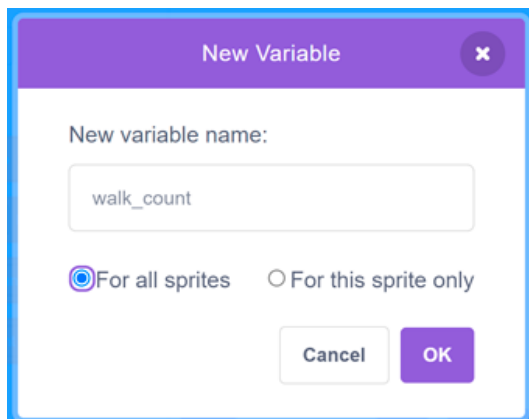


Figure 1.84

The latter option can be useful in more complex projects, but the former option, 'For all sprites', will be sufficient for our purposes in TM111. It is the default option and, unless directed otherwise, you should accept it.

The box offers two options for your new variable:

- For all sprites: this enables scripts for any sprite within your project to use the variable (there is only one sprite here, of course)
- For this sprite only: this restricts use of the variable to the particular sprite you are currently working on.

To do	To note
Click on the 'OK' button.	The palette should look as shown in Figure 1.85.  Figure 1.85
The orange oval block labelled <code>walk_count</code> represents your new variable. It can be dragged to the code area and used in building programs, to report the data it stores.	Notice that to the left of it is a tick box that is ticked by default. If you look at the stage, you should see that the <i>variable watcher</i> shown in Figure 1.86 has appeared.  Figure 1.86 Just as with the watcher for answer, a variable watcher is a visual representation of the value of the variable. Here, <code>walk_count</code>'s watcher shows what has happened as a result of creating the variable <code>walk_count</code> – it shows that a labelled location in memory has been created. The current value of <code>walk_count</code> – that is, the current data stored in this location – is shown as 0, the default value of all new variables in Scratch.

To do	To note
Untick the tick box next to <code>walk_count</code> in the palette and the variable watcher will disappear from	Variable watchers are visible by default when a variable is created and can be a useful development tool, enabling the programmer to keep an eye on the value of a variable as the program runs.

the stage. Then tick the box again to make it reappear on the stage.	However, it may not be appropriate for a variable watcher to be visible to the user of the finished program. For example, think about a number-guessing game in which a variable is used to store a secret number. Moreover, in a program containing many variables it can be distracting for the user to have multiple watchers displayed. So the programmer has to decide, once the program is ready to be used, which variable watchers should be made invisible. We recommend that you keep variable watchers visible whilst creating a program and decide whether to keep them visible once the program is complete.
Click and hold down the left mouse button and drag the watcher to a new position on the stage. Right-click on the watcher.	Variable watchers can be moved around the stage. This can be useful to avoid overlap with sprites. The variable watcher display may be changed to different formats. A 'slider' provides a quick way of setting a variable's value to a number. (Other ways, which you will learn about shortly, are more important.)

(b)

Now that you have created the `walk_count` variable, which blocks enable it to be used in a program? Follow the instructions below.

To do	To note
In the palette there are four blocks underneath the <code>walk_count</code> block, as shown in Figure 1.85. The final two, <code>show_variable[]</code> and <code>hide_variable[]</code>, can be used within a program simply to make a variable watcher appear and disappear from the stage. For example, with a variable holding a secret number in a number-guessing game, you might want to allow the user to see the number only at the end of a game. The first two blocks are more frequently used. Each of these blocks has two input boxes, the first of which offers a drop-down	

menu. This menu holds a list of any variables that have been created. There are two additional items, 'Remove variable' and 'Delete the XXX variable'. As you have only created one variable so far, the drop-down menu has only one variable to choose from, `walk_count`.

Drag the `set[] to[]` block into the code area. Enter 5 into the block's second input box as shown in Figure 1.87 and then click on the block to run it.



Figure 1.87

The variable watcher shows that `walk_count`'s value is now 5; that is, the data stored in `walk_count`'s memory location is now the number 5, as shown in Figure 1.88.



Figure 1.88

To do	To note
Now drag the <code>change[] by[]</code> block into the code area, making sure that it doesn't snap together with your <code>set[] to[]</code> block, and making sure that its input values are set as shown in Figure 1.89.  Figure 1.89	
Click on this block to run it several times, observing the value of <code>walk_count</code> in its watcher.	Each click increases the value of <code>walk_count</code> by 1.
Now enter 2 in the <code>change[] by[]</code> block's second input box and run it a few times, observing the value of <code>walk_count</code>.	<code>walk_count</code>'s value is increased by 2 each time.

Try running the block with negative and decimal numbers as input, such as -5 and 1.5.	The value of <code>walk_count</code> is increased and decreased as you would expect.
Now enter the string 'nonsense' as input to the <code>set[] to[]</code> block and run it.	<code>walk_count</code> now stores the string 'nonsense' as shown in Figure 1.90.  Figure 1.90 Scratch does not pre-determine the type of data that a variable may store. A program may set a variable to any string of characters, be it a number, a word, or something else, for example, '£26.50'. A variable may store different types of data at different points in a program.

(c)

Now use `walk_count` in your program to count the number of times Owl is told to walk, that is, the number of times the `when[a]key_pressed` script runs.

First, drag a `walk_count` block from the palette area to your code area. Note that clicking on it reveals its value, in a bubble.

Over to you

- The first step, common to many situations involving variables, is to initialise the value of `walk_count`; that is, to set it to an appropriate starting value. We want to count the number of times Owl walks, beginning the count when the program starts with the green flag. At this point, Owl won't have walked at all – the count will be zero – so the starting value of `walk_count` should be 0. Add a suitable `set[] to[]` block at the top of your `when_green_flag_clicked` script.
- Next, add a block to the `when[a]key_pressed` script so that each time it is run, `walk_count`'s value increases by 1.
- Finally, add a block to the end of your `when[a]key_pressed` script so that Owl tells the user the number of walks it's now done in the form '5 walk(s), I'm getting tired' for example. You will need to combine a `say[] for[] seconds` block, a `join[][]` block and your `walk_count` block. Start your program with the green flag and then make Owl walk several times using your 'a' key. Restart the program and try it again.

Solution

i. A solution is shown in Figure 1.91.



Figure 1.91

ii. A solution is shown in Figure 1.92.

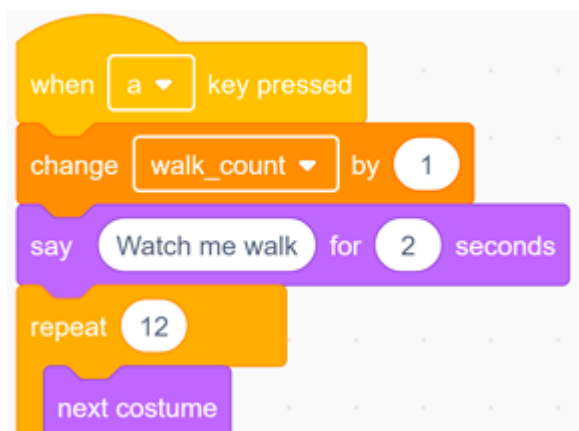


Figure 1.92

The change[]by[] block could be positioned anywhere in this script, except inside the repeat[]{ } block. You might want to experiment to see what happens if it is positioned inside.

iii. A solution is shown in Figure 1.93.

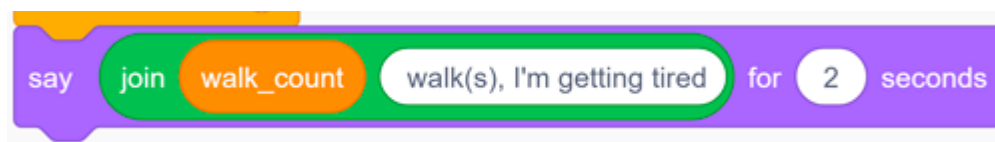


Figure 1.93

Just like a reporter block, a variable block can be nested within an input box of another block. When the nested block runs, the value of the variable is used as input.

A completed program is provided in `Activity_1.14_completed.sb3`.

Further discussion

Initialisation

You have previously learned about initialising visual aspects of a program, such as a sprite's position. In the above activity, you added further initialisation to the `when_green_flag_clicked` script, this time setting the initial value of a variable. Whether a variable needs to be initialised or not depends on its purpose. In the above example, to perform its role in keeping track of the number of walks since the program starts, `walk_count` has to be set to 0 each time the program starts. However, initialisation is not required for all variables, as you will see in the next activity.

1.3.2 Using variables

One common use for a variable, which you have seen, is to keep count of something. Another common use of variables is to deal with user inputs into a program. You have seen that the `answer` block handles data input by the user, but `answer` can only handle one piece of data at a time. What if more than one piece of data from the user needs to be handled by a program? The next activity looks at this.

Activity 1.15 (exploratory)

Open the project `Activity_1.15.sb3` and save it as `Activity_1.15_sol.sb3`.

This project contains the completed program from Activity 1.14.

In this activity, you will amend the `when_this_sprite_clicked` script to add more interaction with the user.

(a)

This script uses the block `answer`, which, as you have seen, is a reporter block that reports data entered by the user in response to an `ask[ ]and_wait` block. Run the `when_this_sprite_clicked` script and enter your name.

The block `answer` reports your name, enabling it to be used by the program. Here, it is used in the subsequent `say[ ]for[ ]seconds` block which, when it is run, takes the name as its input.

In fact, what happens when the user enters data in response to the `ask[ ]and_wait` block is that the data is stored in `answer` and the rest of the program can then use the `answer` block to access the data, just as with a variable. So, just as with variables, we say that `answer` *holds* the user's input, meaning that it provides access to the input data that is stored in a particular place in Scratch's memory.

Let's suppose the `when_this_sprite_clicked` script should now do the following:

- ask the user who they are
- greet them by name

- ask the user for their home town
- tell them that all the best programmers come from their home town.

The script shown in Figure 1.94 achieves this – amend your `when_this_sprite_clicked` script accordingly and try it.

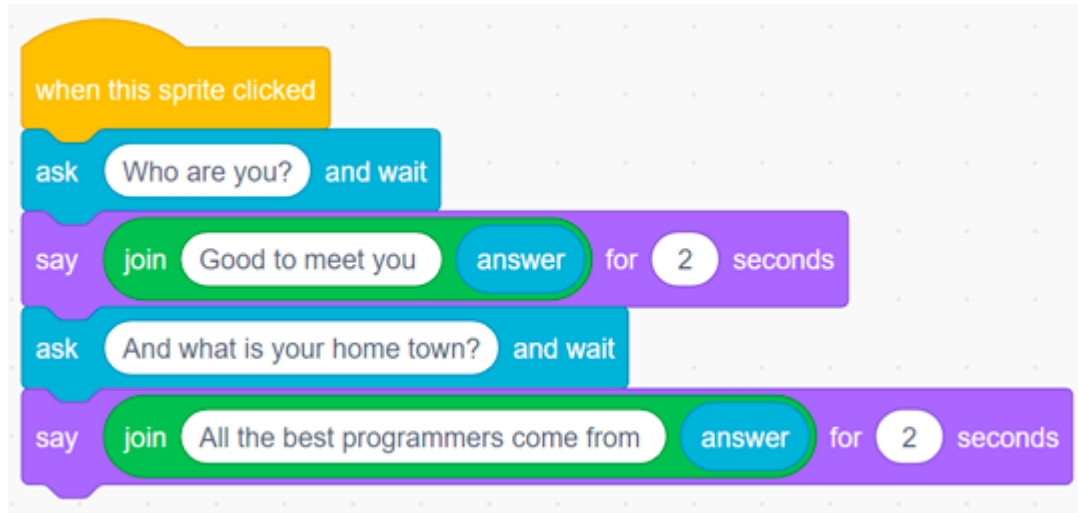


Figure 1.94

(b)

Now suppose your program should instead work as follows:

- ask the user who they are
- ask the user for their home town
- greet them by name
- tell them that all the best programmers come from their home town.

Try rearranging the blocks as shown in Figure 1.95.

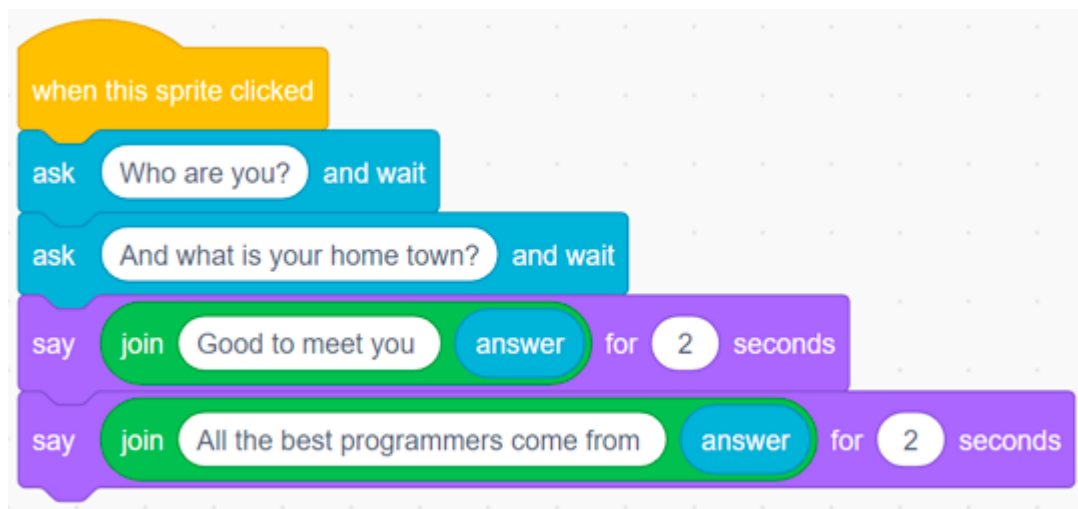


Figure 1.95

But of course there's a problem with this, which you can see if you run the script. Owl greets the user using their home town rather than their name, such as in Figure 1.96.



Figure 1.96

The reason for this can be understood by observing answer's watcher on the stage. (Go to the Sensing palette and tick the box next to answer if necessary.) Run the script and you will see that because its blocks run in sequence from top to bottom, when the user has entered their name, answer's value is that name, such as in Figure 1.97.



Figure 1.97

But with the execution of the next ask[]and_wait block, the user enters their home town. And answer's value becomes that home town, such as in Figure 1.98.



Figure 1.98

This is because the value of answer is always stored at the *same* location in Scratch's memory throughout the execution of a program. So when the user enters new data in response to a subsequent ask[]and_wait block, that new data is stored in that same location, **overwriting** – that is, replacing – any previous data. That previous data is therefore 'lost' – the program can no longer access it – and the new data is instead reported by answer.

This problem can be solved by using a variable to store the user's name.

Create a new variable called name.

How can name help? Well, the first ask[]and_wait block completes its execution when the user enters their name, and consequently answer at this point holds the name. Rather than the program then immediately executing the second ask[]and_wait block, and having the value of answer overwritten by the user's home town, a block can be inserted to store the value of answer – that is,

the user's name – as the value of name, before that value of answer is overwritten. The block is as shown in Figure 1.99.

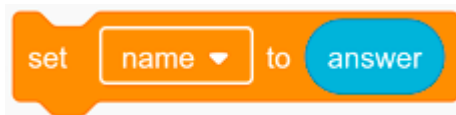


Figure 1.99

When this block is run:

- *first* the value of answer (the user's name) is reported
- *then* name is made to store that value. The user's name is safely stored in the memory location corresponding to name. So even when answer subsequently takes a new value (the home town), the user's name can be found using the variable name.

Over to you

Create this block and insert it as described. (Ensure name is selected as the input to the first input box as you now have more than one variable to choose from.)

Then amend the script so that Owl greets the user as set out at the beginning of part (b).

Run the script and observe how the values of answer and name change as each block in the script runs (tick their tick boxes if necessary to make their watchers visible).

Solution

A solution is shown in Figure 1.100.

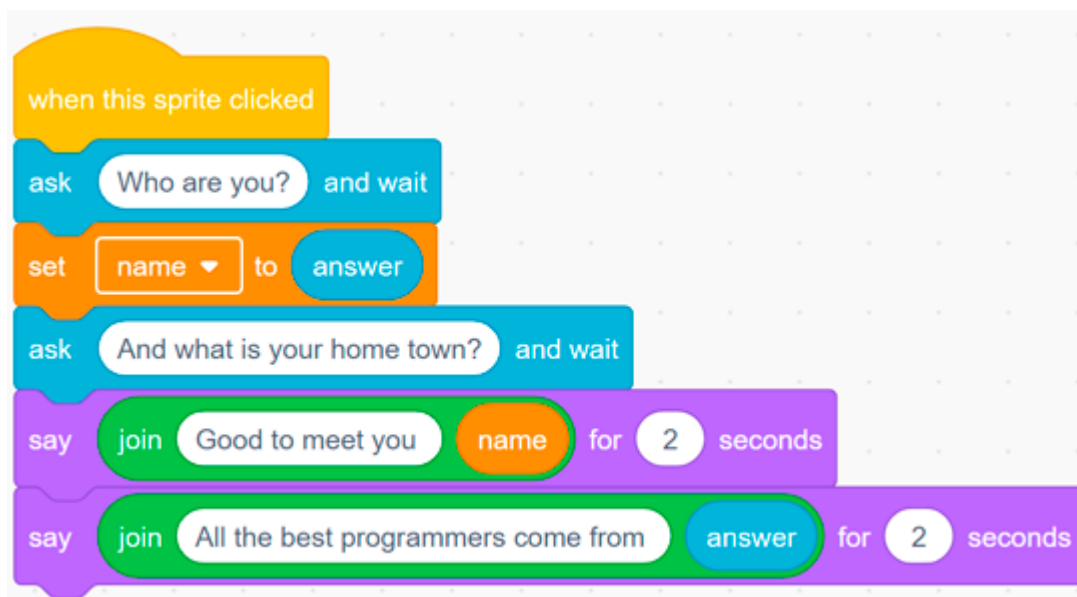


Figure 1.100

(c)

Finally, amend your script as shown in Figure 1.101 so that it uses an additional variable `home_town` (which you will need to create) to store and report the user's home town.

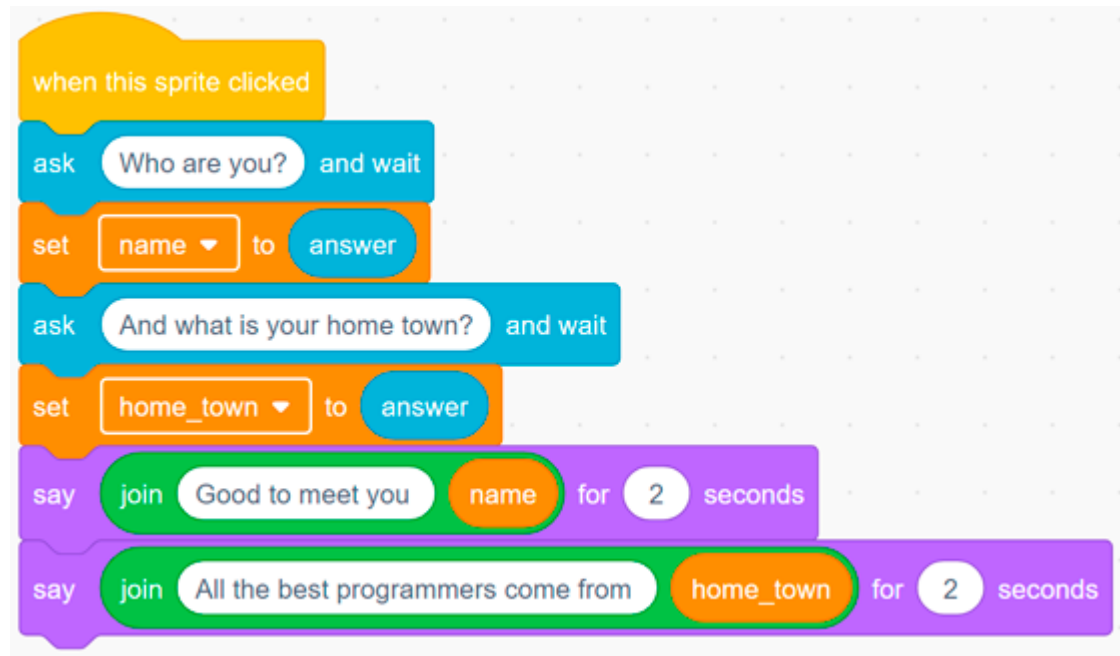


Figure 1.101

Simply run the script and note that it performs just the same task as the original version. The differences between them will be discussed following this activity.

Note: Scratch enables a variable in a program to be changed to another, which is useful if you have used the wrong one. Simply right-click on the variable block within the program and a menu appears with a list of alternative variable names at the bottom, such as that shown in Figure 1.102.

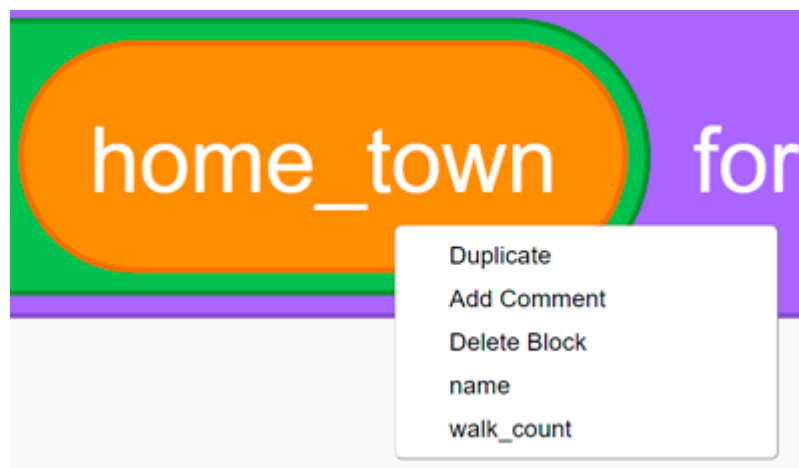


Figure 1.102

A completed program is provided in `Activity_1.15_completed.sb3`.

Further discussion

Initialisation

When using the variable `name` in Activity 1.15, it was not initialised. There was no need to set it at any point within the program to a known, specific value. The `when_this_sprite_clicked` script simply sets `name` to the name input by the user; it doesn't matter what value `name` holds before that point.

This means that, when the script begins, `name` will initially hold the name entered in any previous run. This is not a problem; the script still fulfils its specification.

Some programmers prefer to initialise all variables so that they do not hold any data from previous runs. On the other hand, in a program with multiple variables (and there may be hundreds in some programs) initialising all variables can be tedious and make a program longer and arguably harder to follow. Our policy will be to only initialise a variable where it is necessary for the correct working of a program.

Readability

Imagine returning to the two scripts in Figures 1.100 and 1.101 in a month's time. Which do you think you would find easier to understand?

We think most people would find the script in Figure 1.101 easier to understand – or, in other words, more **readable**.

It is more consistent, handling all input data in the same way. And the use of `home_town` in the final block enables a human reader of the code more easily to see the connection with the home town data entered by the user.

In a script involving several user inputs, or a user input that is used in several places, storing such data in suitably named variables can aid readability. We therefore recommend that in such situations you store user input in suitably named variables.

Some programmers would argue that the script in Figure 1.101 is less efficient than that in Figure 1.100 because it might use more memory to store an extra value, and requires another block to be run, so potentially making the program run more slowly. However, with most programs, the effect of having a few more blocks or a few more variables is negligible, and certainly in our view much less important than the readability of the script.

Why is readability important?

Given that programming languages are primarily a way for people to communicate with computers, why should programs be easy for people to read? There are several reasons why it is important to make your programs readable.

One reason is that readability helps you create programs that work as required. As you build up a program block by block, being clear about what the existing code does helps you get right the code that should

come next.

Even so, programs might still not work correctly at first, and another argument for readability is ease of **debugging**: if your program doesn't run as you expect, you need to be able to find and correct your mistake (**bug**). This involves reading the program and understanding what it does. You will learn more about debugging in subsequent parts.

Activity 1.16 (self-assessment)

Can you think of another reason why programs should be easy for people to read?

Naming variables

In the programming activities above, `walk_count`, `name` and `home_town` were used as variable names. When writing a program, how you name variables is important: as far as possible, you should name them in such a way as to indicate the particular role they have in carrying out the programming task. Choosing meaningful names will help you to remember the purpose of particular variables in your programs and, just as importantly, also help other people to read and understand your programs. For example:

- The name `walk_count` reflects the fact that the value of this variable is the number of times Owl has been made to walk.
- Names such as `var1` or `my_variable` give no clue as to the role played by these variables.

Here are some guidelines to help you choose appropriate variable names in a consistent and meaningful way.

- Although there is no lower or upper limit on the number of characters that can be used in a variable name, very short names such as `a`, `b`, `c` or `ch` should be avoided wherever possible as they are not very informative.
- Names composed entirely of letters of the alphabet are usually the most meaningful to human readers, though it can be appropriate to use digits in names for a number of variables with a similar purpose, such as `total1`, `total2` and `total3`.
- Avoid potential confusion with in-built names; for example, Scratch has a reporter block `answer`, so calling a variable `answer` is not a good idea. Scratch would not be confused by this, but a human might.
- Maintaining a consistent style for variables aids readability. TM111 follows the naming convention of using one or more lower-case English words, separating individual words with underscores: `walk_count`, for example. We recommend that you follow this style for consistency.

Activity 1.17 (self-assessment)

Suppose your program needs a variable to keep count of the number of times a bat has hit a ball.

What might you name the variable?

1.3.3 Conclusion

Activity 1.18 (exercise)

Open the project `Activity_1.18.sb3` and save it as `Activity_1.18_sol.sb3`.

Create a script that makes Owl do the following when the green flag is clicked:

- ask the user 'What's your name?'
- ask the user 'How old are you?'
- tell the user, addressing them by name, that all the best programmers start at their age, such as in Figure 1.103.



Figure 1.103

Reveal comment

In this section, you have been introduced to the programming mechanism of a variable, and how variables are handled in Scratch programs.

1.4 Sprite interactions and concurrency

In this section, you will progress from working with programs involving a single sprite to working with programs involving more than one sprite. You will learn how sprite actions may be coordinated. You will also see how a Scratch program may do several things at the same time.

1.4.1 Sprite interactions

Activity 1.19 (exploratory)

Open the project `Activity_1.19.sb3` and save it as `Activity_1.19_sol.sb3`.

This project contains a complete program that we have created in which two sprites, Penguin and Polar Bear, tell a joke.

Note: Video 1.4, which you will watch in the next section, shows you how to bring different sprites into a project.

In this activity, you are simply going to explore this program. You will amend it in Activity 1.20.

Start the program with the green flag and then click on the penguin image on the stage. You will see that the penguin and the polar bear tell the joke, line by line, in turn.

Let's look at how their 'speech' is coordinated.

(a)

Select Penguin using its thumbnail below the stage, as shown in Figure 1.105.

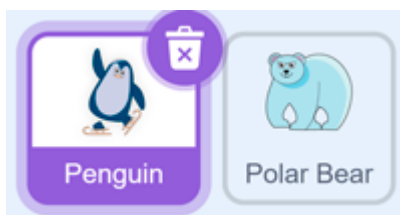


Figure 1.105

Penguin's when_green_flag_clicked script simply greets and instructs the user.

When Penguin's image on the stage is clicked on, the when_this_sprite_clicked script, shown in Figure 1.106, initiates the joke. The say[]for[]seconds blocks correspond to the lines of the joke spoken by Penguin. The broadcast[]and_wait blocks are the key to Polar Bear's involvement.

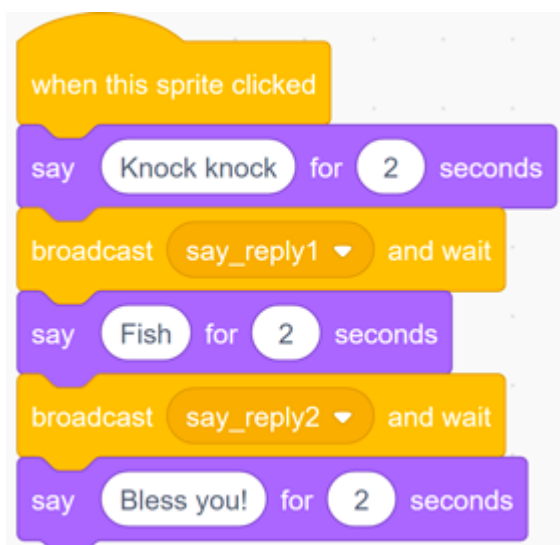


Figure 1.106

Consider the first such block: broadcast[say_reply1]and_wait. The broadcast[]and_wait block is found in the Events palette. It sends, or **broadcasts**, an 'invisible' message with the name in

its input box – here, `say_reply1` – rather like a theatre cue. Other scripts may then run in response to this message; we will describe shortly how this is arranged.

Note: We use the same TM111 naming convention for messages as for variables.

The 'wait' part of the `broadcast[]and_wait` block means that the script this block is part of will pause execution until all responding scripts have finished running.

(b)

Now select Polar Bear using its thumbnail below the stage.

You will see that it has two scripts headed with `when_I_receive[]` hat blocks. We call such scripts **when_I_receive[] scripts**.

One of the scripts is shown in Figure 1.107. Its hat block's input value is `say_reply1`, which is the name of the message broadcast by Penguin's first `broadcast[]and_wait` block.

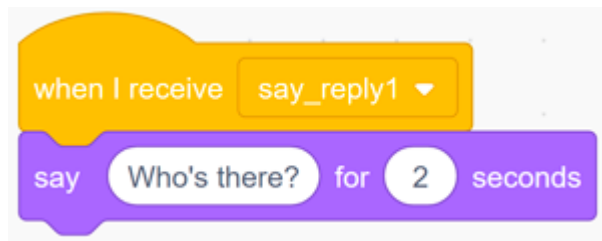


Figure 1.107

When a message is broadcast, all `when_I_receive[]` scripts which have the name of that message as input to their hat block start running.

So when the message `say_reply1` is broadcast, the script shown in Figure 1.107 runs and Polar Bear speaks.

This is how Polar Bear says its line of the joke at the right time, directly after Penguin's first line.

(c)

Turning back to Penguin's `when_this_sprite_clicked` script, note that Penguin's next line is implemented in the `say[]for[]seconds` block immediately after the `broadcast[say_reply1]and_wait` block. So, once Polar Bear has finished its line of the joke, Penguin speaks again.

After that comes another `broadcast[]and_wait` block – this one with input `say_reply2`. This prompts Polar Bear to deliver its next line, via its corresponding `when_I_receive[]` script shown in Figure 1.108.

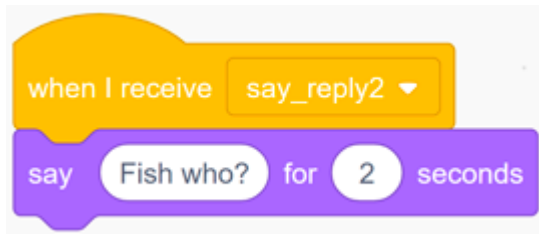


Figure 1.108

(d)

To understand the importance of the 'wait' part, with Penguin selected go to the Events palette and drag a broadcast[] block to the code area. This block broadcasts a message, just as the broadcast[] and _wait block does, but it doesn't pause the execution of the script it is part of.

Swap it into Penguin's script as shown in Figure 1.109.

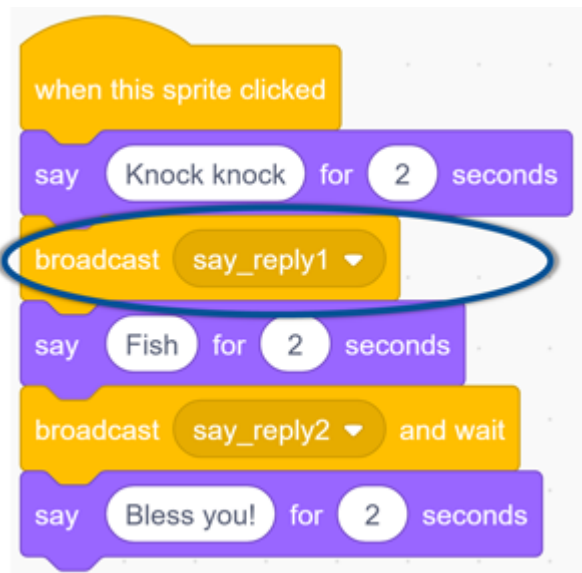


Figure 1.109

Now when this script is run, you will see the problem: Polar Bear's speech overlaps with that of Penguin: Penguin does not wait for Polar Bear's response before continuing with the joke. The broadcast[] and _wait blocks are required so that Penguin does not deliver its next line until Polar Bear has responded.

Revert to the original version of the script, as shown in Figure 1.106.

(e)

Finally, run this part of the program by clicking on Penguin. Do this twice: the first time observing Penguin's code area and the second time observing Polar Bear's code area. Note that the currently executing script is highlighted yellow.

Further discussion

Sprite interactions

A sprite can initiate action in another sprite or sprites by sending a message using the broadcasting mechanism. Just as the user clicking on the green flag is an event, so the receipt by a sprite of a broadcast message is a kind of event, though it is an 'internal' one, not initiated directly by the user. But similarly, a sprite must have a suitable script in order to respond to the event of a broadcast message being received.

Care must be taken to ensure, where necessary, that scripts pause whilst others are running by using `broadcast[]` and `wait` blocks as appropriate. In other words, sprite interactions may need to be carefully coordinated.

1.4.2 Concurrency

In the next activity, you will add some finishing touches to the program from Activity 1.19, and in doing so will learn a little about how to make programs appear to do more than one thing simultaneously.

Activity 1.20 (exploratory)

Open the project `Activity_1.20.sb3` and save it as `Activity_1.20_sol.sb3`.

This project contains the completed program from Activity 1.19.

(a)

Create and run the script for Penguin shown in Figure 1.110. The `forever{}` block is in the Control palette.

Note: Remember to press the red stop icon to stop scripts running.

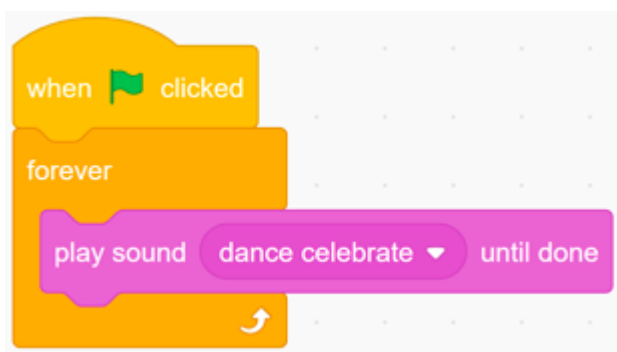


Figure 1.110

The `forever{}` block runs the blocks inside its jaws indefinitely (or at least until the red stop icon is clicked or the program or computer are shut down). The `forever{}` block is another way, in addition to the block `repeat[]{}{}`, in which repetition or looping is achieved in Scratch.

Note: Part 4 looks at looping in more detail.

This script plays a sound – some music – repeatedly, indefinitely.

(b)

Penguin now has two `when_green_flag_clicked` scripts.

(i)

See what happens when you run Penguin's *other* `when_green_flag_clicked` script by clicking on it.

Just that single script is run, not the newly-added `when_green_flag_clicked` script.

(ii)

See what happens when you start the program with the green flag.

Now *both* `when_green_flag_clicked` scripts run at once.

(c)

Over to you

Finish by creating another `when_green_flag_clicked` script for each of Penguin and Polar Bear, to make each do their own dance throughout the duration of the program. Use the blocks shown in Figure 1.111 for each.

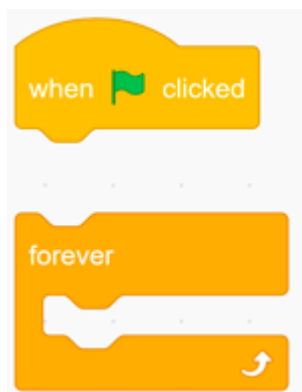


Figure 1.111

You can use your imagination; there is no strict specification, but each dance should involve:

- animation via costume changes
- position and/or direction changes
- some change to the sprite's graphic effects, using the Looks block shown in Figure 1.112, which you should explore



Figure 1.112

- initialisation of the sprite's graphic effects using the Looks block shown in Figure 1.113, which you should explore

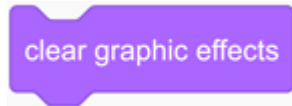


Figure 1.113

- initialisation of the sprite's position and/or direction, if these aspects are changed during the sprite's dance.

Run your program; that is, start it with the green flag and then click on Penguin. Watch the scripts executing in the two code areas. Click on the red stop icon to stop.

Solution

Here are two example scripts.

- In Penguin, the script in Figure 1.114 makes it appear to 'rock' from side to side in a sort of skating movement, changing colour as it goes. Note that we have initialised Penguin's direction, since this is changed during its dance. Note the use of the `wait[]seconds` block, to make the 'rocking' motion visible.



Figure 1.114

- In Polar Bear, the script in Figure 1.115 makes it appear to 'lumber' from side to side, with a whirl effect for added visual interest. Note that we have initialised Polar Bear's position, since this is changed during its dance.

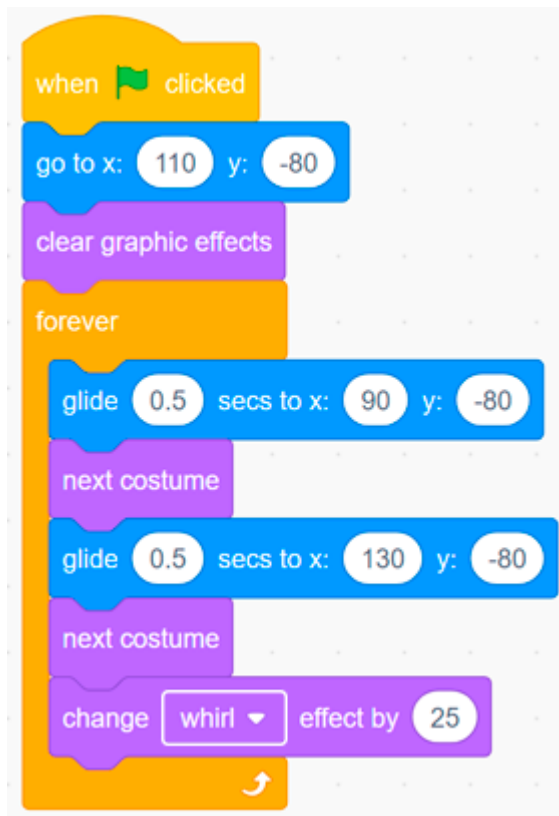


Figure 1.115

When the program starts, you should see:

- all `when_green_flag_clicked` scripts running at the same time
- Penguin's `when_this_sprite_clicked` script running in response to the user clicking on Penguin
- Polar Bear's `when_I_receive[ ]` scripts running in response to the messages broadcast from within Penguin's `when_this_sprite_clicked` script.

Further discussion

Multiple sprites

A Scratch program may contain any number of sprites, each of which has a different code area. A script created for a sprite, that is, in that sprite's code area, instructs Scratch to carry out the script instructions *in relation to that sprite*.

Concurrent execution

A Scratch program that is required to do several things at once may involve several scripts all executing concurrently, each implementing a particular task. As mentioned earlier, in fact what happens is that Scratch switches rapidly between the scripts.

Flow of execution

Whenever a script is running, it is highlighted yellow in its code area. This provides a way of tracking the **flow of execution** through a program – that is, the order in which its parts are run – and can be particularly useful in ensuring that sprite interactions are working correctly. Thinking about the required flow of execution as you are creating a program helps you build up a mental picture which can, in turn, help you decide how to implement the various tasks. For example, for the program in Activity 1.20 you might have in mind something like Figure 1.116.

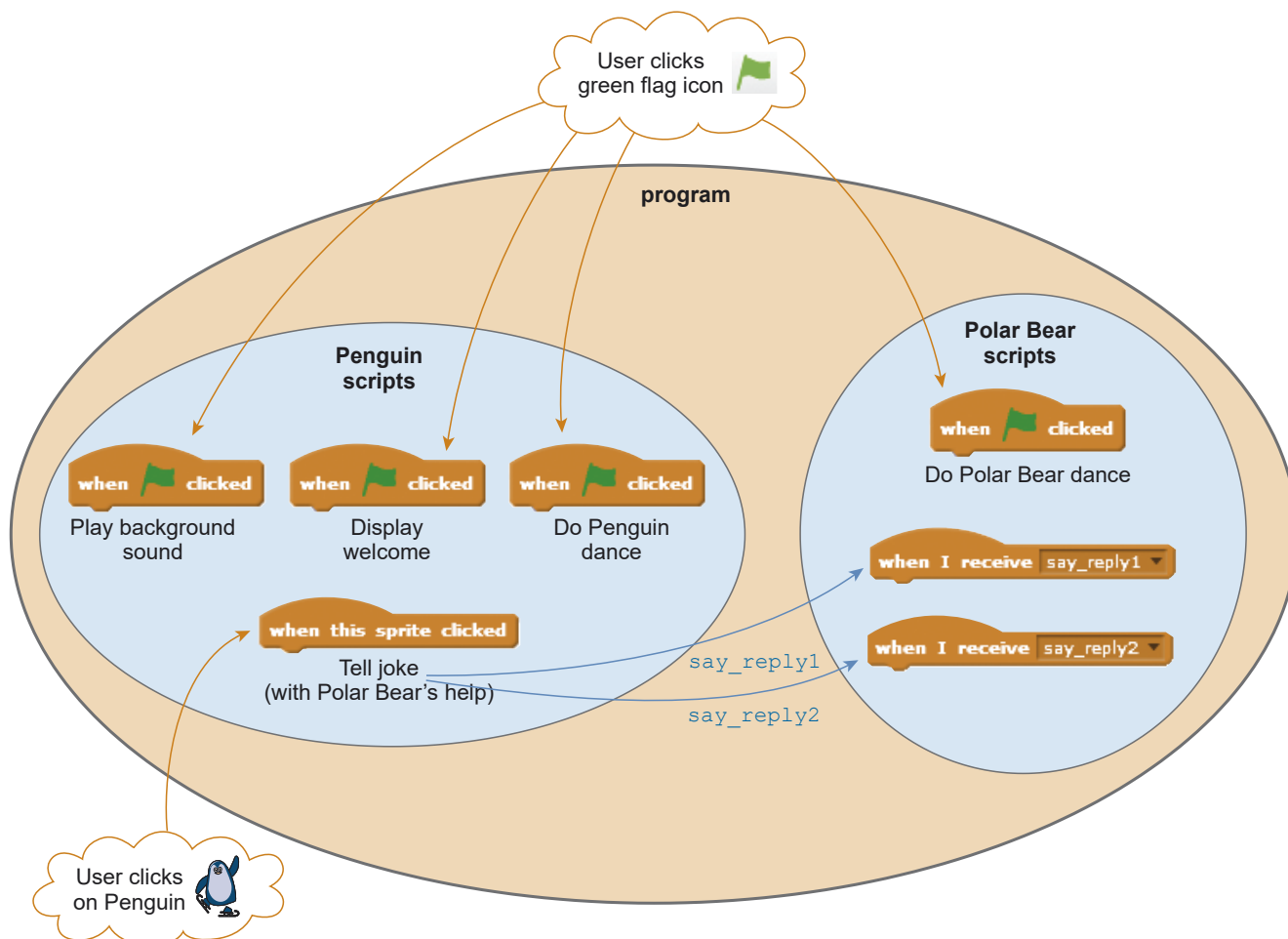


Figure 1.116

[Maximise](#)

Running scripts during program creation

Running a script by clicking on it is a development tool, that is, something to help the programmer when creating the program. When you have several scripts in a program, you as the programmer can run each of them individually in this way, but you should also make sure you run them as a user would (for example, by clicking the green flag icon or clicking on a sprite) to ensure that they interact properly together and that their overall effect is as intended.

1.4.3 Conclusion

Activity 1.21 (exercise)

Open the project Activity_1.21.sb3 and save it as Activity_1.21_sol.sb3.

This project contains sprites Button and Lightbulb. Lightbulb has two costumes, one showing the lightbulb on and the other showing the lightbulb off.

(a)

Create a script for Button and a script for Lightbulb so that when the user presses (that is, clicks on) the button on the stage, the lightbulb switches on (if it is off) or off (if it is on). Use the `next_costume` block to change the appearance of the lightbulb.

Note: To enter a new message name as input to any of the blocks `broadcast[ ]`, `broadcast[ ]and_wait` and `when_I_receive[ ]`, click on the 'New message' option in the drop-down menu in its input box.

(b)

Now make the button seem more realistic by appearing momentarily faded when clicked on, with the normal (unclicked) appearance shown in Figure 1.117 and the faded (clicked) appearance shown in Figure 1.118.



Figure 1.117



Figure 1.118

Use the blocks:

- `change[ ]effect_by[ ]` (explore the effect of the input value ghost)
- `clear_graphic_effects`
- `wait[ ]seconds.`

Reveal comment

In this section, you have started working with programs that involve more than one sprite, learning how sprite actions may be coordinated. You have also learned how a Scratch program may do several things at the same time.

1.5 Fun and games

In this section, you will undertake an exercise on a musical theme which enables you to practise some of the things you have learned about in this part. First you need to know a few more facts about working in Scratch.

Activity 1.22 (exploratory)



Watch the following:

- Video 1.3, which demonstrates some ways of working with sprites, such as creating a new sprite and duplicating an existing one
- Video 1.4, which looks at working with costumes, backdrops and sounds.

Sounds and graphics

From now on, you'll be working, from time to time, on projects in which you might import or create your own sounds and graphics. Be aware that perfecting them is not one of TM111's objectives: we expect only that sounds and images adequately convey visual and auditory information. Take care – it can be very easy to spend too much time on perfecting sounds and images at the expense of time spent learning about programming.

Activity 1.23 (exercise)

There are no detailed instructions for this activity because we want you to feel free to achieve the specification however you wish. If you have prior programming experience, for example, you may find you can create the program quite quickly. However, we expect that many people will need more guidance, and so we have structured this as a series of hints at the end of the activity. Read as many or as few of them as you need. They suggest some possible ways to create this program, but there are others.

So bear in mind that your final program may be fine, even though it might not match our completed program. If you are not confident that yours is correct, you might ask your tutor, or fellow students in a forum. If you have trouble arriving at a working program within an hour or so, even after you have worked through the hints, ask your tutor for help.

You are going to create a small program in which a musician introduces their instruments. There will be a human sprite, Musician, and three instrument sprites. For illustration, we will use the scenario shown in Figure 1.122, in which Musician is accompanied by sprites Xylophone, Violin and Drums, but you are free to choose whichever Musician and instrument sprites you wish.



Figure 1.122

We will describe the program in two parts.

Starting with the green flag

When the program starts with the green flag, Musician should ask the user to give them a name, as shown in Figure 1.123. They should then give the user the instruction shown in Figure 1.124.



Figure 1.123



Figure 1.124

Clicking on Musician

Then when Musician is clicked on, Musician should introduce themselves, using the name the user entered, such as the example shown in Figure 1.125.



Figure 1.125

Then each instrument, in turn, should act as follows:

- the instrument tells the user that the musician can play that instrument, such as the example shown in Figure 1.126



Figure 1.126

- the instrument plays an appropriate sound for that instrument.

(a)

Open a new project and save it as `Activity_1.23_sol.sb3`.

A new project is always the default project, containing the Scratch cat. You should now simply delete the cat by selecting Sprite1 at the right-hand side of the stage and then clicking the dustbin icon. This is the standard way to start creating a program involving your own sprites.

Bring in your new sprites now, making sure that each instrument is equipped with a corresponding sound. Choose a sprite from a library, paint a new sprite or upload a sprite from your computer (for example, an image that you have found online). If your sprite does not come equipped with a suitable sound, then record one, choose one from a library or upload one from your computer.

Note: Your Musician sprite will need to be clicked on. You should ensure that it has a solid costume as clicks on transparent parts of a costume are not recognised.

Note: If you are short of time, use the same sprites as us, which are in the TM111 sprite library, supplied in the project file.

(b)

Now create scripts for each of your sprites to fulfil the specification of the program given above.

(c)

Then explore the blocks from the Looks palette shown in Figure 1.127.



Figure 1.127

Use these blocks to amend the program so that:

- when the program starts with the green flag, only the musician is visible on the stage
 - each instrument appears when it is time for it to act and remains visible for the remainder of the program's execution.
-

(d)

If you have time, extend the program, inventing your own specification. Use your imagination. Will you add more instruments? Animate instruments? Add a background?

Hints

(b)

What scripts?

You could start by creating suitable hat blocks in each sprite:

- Musician must do something when the green flag is clicked, so you'll need a `when_green_flag_clicked` script for Musician
- Musician must also do something when clicked on, so you'll also need a `when_this_sprite_clicked` script for Musician
- each instrument sprite must do something when its turn comes to act. It can be prompted by a broadcast message. So you'll need a `when_I_receive[ ]` script for each instrument sprite.

Planning interactions

Now think about how the scripts will interact, and keep this in mind as you create the scripts themselves. The instrument sprites will need to act in turn when Musician is clicked on. One way to coordinate the actions is to have the `when_this_sprite_clicked` script prompt each sprite to act, as illustrated in Figure 1.128.

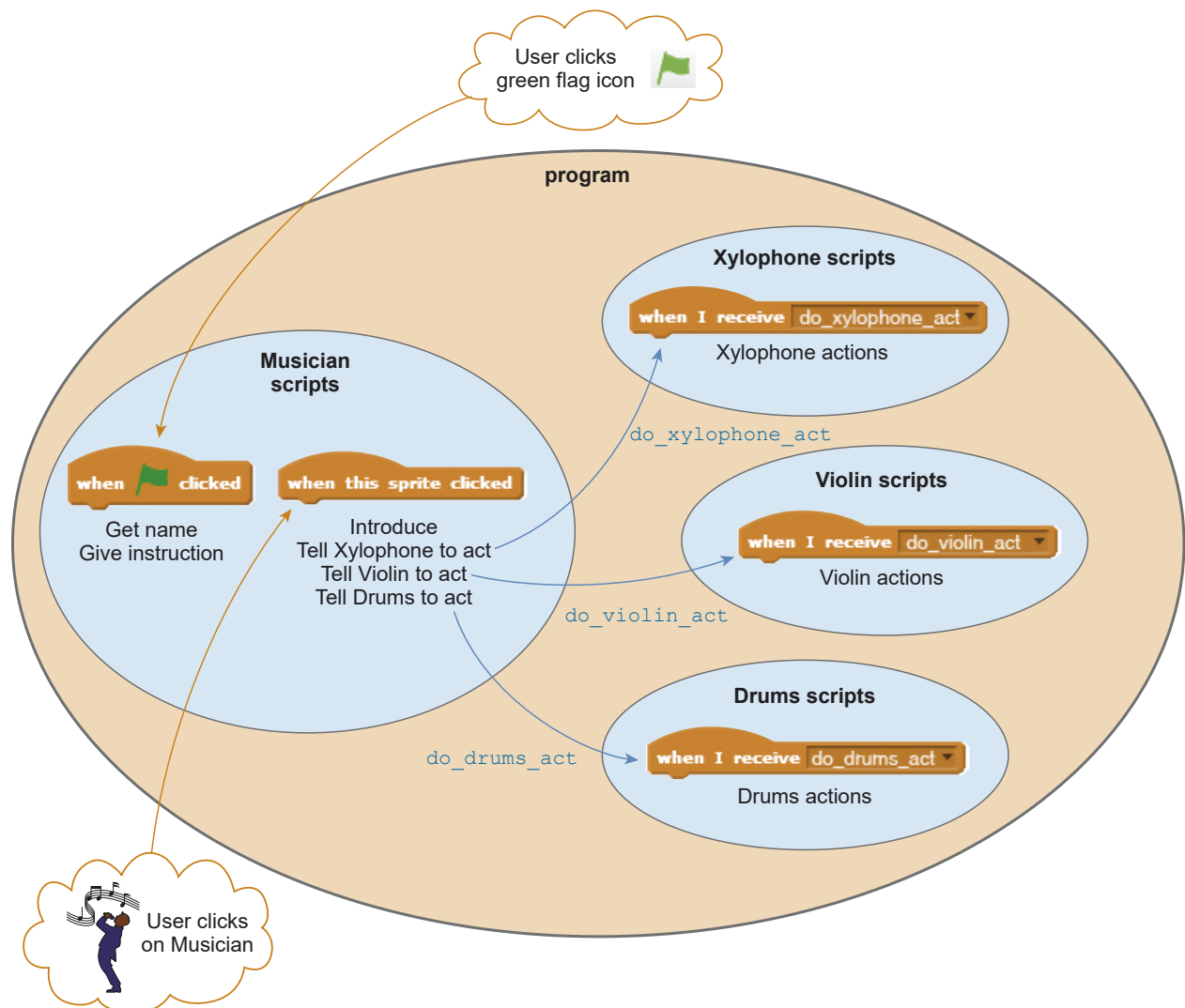


Figure 1.128

[Maximise](#)

Creating the scripts

In Musician's `when_green_flag_clicked` script, you could:

- use an `ask[]and_wait` block to obtain the musician's name
- store the musician's name in a suitable variable
- use a `say[]for[]seconds` block to make the musician tell the user 'Click on me'.

In Musician's `when_this_sprite_clicked` script, you could:

- use nested `say[]for[]seconds` and `join[][]` blocks, along with the variable storing the musician's name, to make Musician introduce themselves

- broadcast a sequence of messages to instruct each instrument to act in turn, waiting if necessary for the act to finish. Ensure each message is named identically to the message in the corresponding `when_I_receive[]` script.

In each instrument sprite's corresponding `when_I_receive[]` script, you could:

- use nested `say[]for[]seconds` and `join[][]` blocks, along with the variable storing the musician's name, to make the instrument tell the user that the musician can play the instrument
- use a `play_sound[]until_done` block to play the corresponding instrument sound to completion.

(c)

- Each instrument sprite should initially be invisible. You could create a `when_green_flag_clicked` script for each instrument sprite that simply contains a `hide` block.
- To make each instrument sprite appear at the beginning of its act, you could add a `show` block at the top of its `when_I_receive[]` script.

A completed program is provided in `Activity_1.23_completed.sb3`.

1.5.1 Conclusion

In this section, you have practised your programming skills by working more independently on an exercise. In each of Parts 2–5 there will be a similar final exercise whose context is some kind of entertainment – for example, an animation or game – which we hope you will find enjoyable to work on as well as being a useful consolidation and development opportunity.

Summary

In this part, you have learned about the main features of the Scratch environment. You have started creating small programs in Scratch, learning along the way about some different kinds of blocks and how they can be used together to build scripts to fulfil simple specifications.

You have seen how different types of script can be created to correspond to different events that may take place; how there may be more than one script that corresponds to the same event (clicking on the green flag, for example), and which therefore execute concurrently; and how to use broadcasting to achieve interaction between sprites.

You have learned how user input can be handled; about the use of strings in Scratch; and about the basics of variables.

The scripts you have created mostly involve blocks running in sequence, from top to bottom, but you have also had a small taste of looping structures in Scratch, which enable blocks to be executed repeatedly. We

will use such structures from time to time from here on, but you will learn more about the concepts behind them in Part 4.

In Part 2 you will gain more experience in using variables to work with data. You will learn more about strings, and about how to work with numbers in Scratch. You will also learn about a new concept, that of a list, which enables programs to work efficiently with collections of data.

Having completed this part, you should be able to:

- describe the tasks performed by a small Scratch program involving sequences of blocks
- amend or create a small Scratch program involving sequences of blocks to meet a specification.

Answers to self-assessment activities

Activity 1.6

The output would be the text '5 seconds to liftoff...' displayed in a speech bubble for 5 seconds.

Activity 1.9

- A has coordinates $(-240, 0)$. B has coordinates $(0, 180)$.
- The sprite ends up at position $(-50, 90)$.
- Increase x position by 290; decrease y position by 50.

Activity 1.10

Its x position is unchanged and its y position decreases by 50.

Activity 1.12

Owl's position would need to be initialised to $(0, 0)$ before the walk, such as in Figure 1.129.

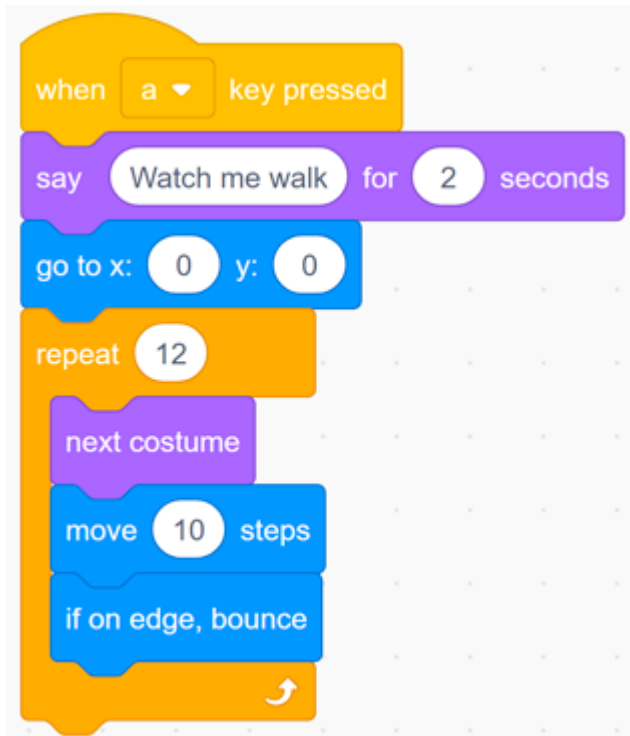


Figure 1.129

Activity 1.16

Another reason is that you may wish to modify an existing program – either your own, at a later date, or one created by someone else – rather than create one afresh in order to carry out a particular task. Again, it is important that you understand what the existing program does.

Activity 1.17

Something like `number_of_hits` would be appropriate.

Acknowledgements

Grateful acknowledgement is made to the following sources:

Figures 1.1 to 1.52 and 1.55 to 1.129: Licensed under Creative Commons Attribution-ShareAlike 2.0 license. <http://scratch.mit.edu>.

Scratch is developed by the Lifelong Kindergarten Group at the MIT Media Lab. See <http://scratch.mit.edu>.

Every effort has been made to contact copyright holders. If any have been inadvertently overlooked the publishers will be pleased to make the necessary arrangements at the first opportunity.